Data Mining 2 Project

Camilla Maffeis Giulia Fabiani

A.Y. 2024/2025

CONTENTS

1 DATA UNDERSTANDING AND PREPARATION

The goal of this project is to analyze two datasets providing different kinds of information about movies, TV shows, and other forms of visual entertainment.

In the following section, we'll focus on the tabular dataset, while the overview about the time series dataset composition and preparation can be found in Section 6.

This dataset is an extension of the one proposed in DM1: the records include key information about titles, insights into critical aspects such as awards and reviews, as well as ratings and further data about the title's production.

1.1 Data Semantics, Duplicates and Train/Test Split

The tabular dataset contains 149,531 records and 32 attributes; the nine attributes that were added to the original set from the DM1 dataset are reported in Tab. 1. This makes a total of nine categorical attributes, three binary attributes and 20 numerical attributes. Among the latter, only *runtimeMinutes* and *averageRating* can be classified as continuous, as the others mostly represent counts.

A first data quality check revealed only ten genuine duplicates, i.e. records sharing all feature values, which we dropped.

We then split the dataset, reserving 70% for training, and the remaining 30% for testing. As to avoid data leakage, we performed all subsequent data exploration exclusively on the training set. At the end of the data preparation stage, we applied all transformation and cleaning steps to the test set as well.

1.2 First Global Exploration of the Training Set

A first global analysis of the dataframe's summary and descriptive statistics revealed missing values for *endYear* (100,729), *runtimeMinutes* (28,185), *countryOfOrigin* (27,961) and *genres* (1,887). Since most records are missing an *endYear*, and the feature is strongly correlated to *startYear*, we dropped it.

Given that the great majority of records had no information about sound mixes, we decided to drop this feature as well.

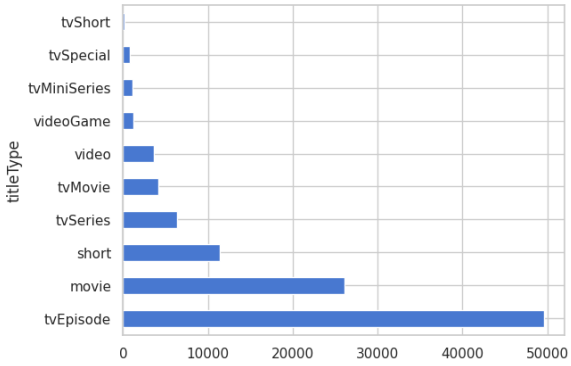


Figure 1: Distribution of *titleTypes*.

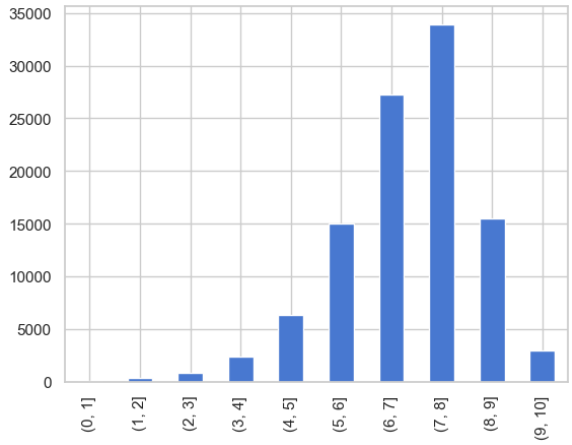


Figure 2: Barplot for *rating*.

bestRating, *worstRating*, and *isRatable* were once again confirmed to be uninformative, as they only take one value for all records (10, 1, and 1, respectively), and were therefore dropped. We also dropped *ratingCount*, as it was strongly correlated to *numVotes*, and *isAdult*, because it's redundant w.r.t. the *adult* category in *genres*.

Checking the values for the only remaining original binary attribute, *canHaveEpisodes*, we noticed that only two records with a *titleType* different from *tvSeries* and *tvMiniSeries* have a positive value. We corrected them: the first had been registered as a *tvMovie* instead of *tvSeries*, while the second was actually a short with no episodes.

Lastly, we removed the string `'\\N'`, which was prepended to all lists of *regions*, making the feature consistent with *numRegions*.

1.3 Distribution of Categorical Variables and Statistics

In this subsection, we studied the univariate and bivariate distribution of categorical variables, which all turned out to be unbalanced.

Feature	Description	Type
castNumber	Total Number of Cast individuals present within the IMDb title page.	Discrete. Ratio-scaled.
companiesNumber	Total Number of companies that worked for the title.	Discrete. Ratio-scaled.
regions	The regions for this version of the title. Some titles can belong to more than one class.	Categorical. 171 unique classes.
averageRating	Weighted average of all the individual user ratings.	Continuous. Ratio-scaled.
externalLinks	Total Number of External Links the title has within the IMDb page.	Discrete. Ratio-scaled.
quotesTotal	Total Number of quotes the title has within the IMDb page.	Discrete. Ratio-scaled.
writerCredits	Total number of writer credits of the title.	Discrete. Ratio-scaled.
directorCredits	Total number of director credits of the title.	Discrete. Ratio-scaled.
soundMixes	Technical specification of the sound mixes available for the title. Some titles can belong to more than one class.	Categorical. 59 unique classes.

Table 1: New features in the extended dataset.

Differently from the DM1 dataset, the most popular ***titleType*** is *tvEpisode* (47% of the records), followed by *movie* (25%) and *short* (11%); *tvSpecial* (0.79%) and *tvShort* (0.17%) are the classes with the lowest support (cf. Fig 1). With regard to ***genres***, the two most popular classes are by far *Drama* and *Comedy*, although the distribution depends on *titleType*. ***rating*** follows a left-skewed distribution, with the mode being the *[7,8)* class (Fig. 2). The most popular ***countryOfOrigin*** and ***region*** is the US, but both features have a substantial amount of missing values: *nan* is the second most popular value for both.

We dropped *soundMixes*: not only are most of its 59 unique values very rare, it has too many missing values to provide useful information.

1.4 Distribution of Numeric Variables and Statistics

Before analyzing numeric variables, we added the same features we had added to the DM1 dataset:

- *numCountriesOfOrigin*: the number of countries listed for the record for the variable *countryOfOrigin*.
- *numGenres*: the number of genres listed in *genres*. For the training set, the values range from 1 to 3.
- *reviewsTotal*: the sum of *criticReviewsTotal* and *userReviewsTotal*.
- *criticReviewsRatio*: the proportion of review from critics (*criticsReviewTotal*) on the total number of reviews of a record (*reviewsTotal*). For unreviewed records, it is set to zero.
- *awardsAndNominations*: the sum of *awardWins* and *awardNominationsExcludeWins*.

A pairplot of all numeric variables, with KDE graphs on the diagonal, allowed us to gain a global view of both the features' univariate distribution and their pairwise correlations. Given the size of the dataset, we resorted to using a random sample of 20,000 records. The great majority of the variables has a right-skewed distribution (*writerCredits*, *numRegions*, *awardWins*, *totalImages*, *criticReviewsTotal*, *runtimeMinutes*, *userReviewsTotal*, *totalCredits*, *totalVideos*, *castNumber*, *numVotes*, *quotesTotal*, *awardNominationsExcludeWins*, *companiesNumber*, *externalLinks*, *awardsAndNominations*, *numCountriesOfOrigin*, *reviewsTotal*). *averageRating*, of course, follows the same left-skewed distribution observed for its binned version, *rating* (cf. Fig. 2).

We observed that some records have an implausibly long runtime, probably due to the inconsistent strategy adopted for the computation of the runtime for serialized content, for which sometimes the total runtime of a series is registered, rather than the average episode runtime. In order to avoid this problem, we removed the runtime of all *tvSeries*, *tvMiniSeries*, and *tvEpisodes* for which the value exceeded the 3rd quartile + 1.5 interquartile range for its specific category (105, 382.5 and 84 minutes, respectively); a total of 1560 records in the training set were affected.

A similar problem was found for *writerCredits*: records with suspiciously high values are likely TV episodes for which all writers of the entire TV series were erroneously credited. Therefore, we decided to remove the values for *writerCredits* from 2100 *tvEpisodes*, as they exceeded the 3rd quartile + 1.5 interquartile range (8.5 credits).

1.5 Variable Transformation, Pairwise Correlations and Elimination of Variables

We log-transformed all right-skewed variables, and used the Shapiro-Wilk test to determine whether they now resembled a normal distribution. None of them did, therefore we used Spear-

Original feature	Boolean version	% True
awardWins	hasAwards	8.16
awardsAnd Nominations	hasAwardsOr Nominations	12.25
totalVideos	hasVideos	7.93
criticReviewsTotal	hasCriticsReviews	24.88
userReviewsTotal	hasUserReviews	36.98
directors Credits	moreDirectors Credits	9.89
quotesTotal	hasQuotes	13.86
numCountries OfOrigin	moreCountries OfOrigin	5.05

Table 2: Binarization of numeric features.

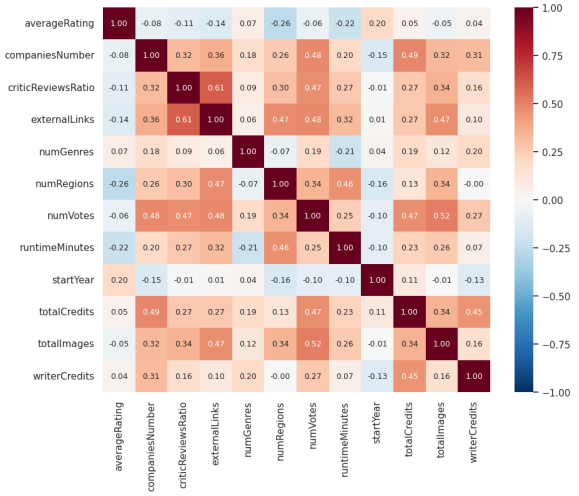


Figure 3: Correlation heatmap after attribute removal.

man's correlation coefficient in our correlation analysis.

We removed the following features because they were highly correlated to another feature: *awardNominationsExcludeWins* (0.86, correlated with *awardsAndNominations*), *castNumber* (0.74, with *totalCredits*). Since all features regarding reviews and *numVotes* were highly correlated with each other, we dropped them all with the exception of *numVotes*, which has the least skewed distribution, and *criticReviewsRatio*, which offer insights of a different nature. We however decided to conserve information about the presence/absence of reviews by users and critics with the creation of two new boolean features: *hasUserReviews* and *hasCriticsReviews*.

Moreover, we binarized features which overwhelmingly took on the value 0 (or 1, in the case of *numCountriesOfOrigin* and *directorsCredits*). The results of these transformations are reported in Table 2.

The total number of final features is 27; they are listed in Tables 3 and 4. Among the remaining numeric features, the heatmap in Fig. 3 shows that *externalLinks* exhibits a moderate to high positive correlation with *criticReviewsRatio* (0.61) and a low to moderate correlation with *numRegions*, *numVotes* and *totalImages* (around 0.47), while *numVotes* is also mildly correlated to *totalImages*, *totalCredits*, *criticReviewsRatio*, and *companiesNumber* (between 0.47 and 0.52).

1.6 Handling Missing Values

At this point, we still have missing values for *countryOfOrigin*, *region*, *genres*, *runtimeMinutes* and *writerCredits*; some were preexisting, others were removed as detailed in Subsec. 1.4. Since the distribution of *runtimeMinutes* depends on *titleType*, and the same holds true for *writerCredits*, we imputed their missing values using the median value within each respective class.

On the other hand, we chose not to replace missing values for *genres*, *region* or *countryOfOrigin*. They can easily be handled when performing the encoding for our methods without introducing bias, therefore we'd rather preserve the original distributions and correlations.

1.7 Clustering

As part of the data understanding phase, we conducted hierarchical clustering on the dataset, using all numeric attributes after normalization performed with a Robust Scaler.

Since the size of the dataset made it unfeasible to perform hierarchical clustering directly because of the cost of computing distances among all records, we firstly reduced our dataset to a more manageable 10,000 records, running k-means clustering with k=10,000.

We then performed hierarchical clustering on the centroids of K-Means clusters, experimenting with different linkage methods and inspecting the resulting dendrograms. We found that using average linkage yielded the most satisfactorily results. We then cut the dendrogram at an appropriate threshold and, after propagating cluster labels back to the entire dataset, analyzed the composition of the clusters.

We found 13 clusters of varying sizes, with the top cluster grouping one fifth of the entire dataset and the top four clusters grouping half of the dataset. Five clusters contained less than 5,000 records each.

Feature	Description	Type
runtimeMinutes	Primary runtime of the title, in minutes.	Continuous. Ratio-scaled.
startYear	Represents the release year of a title. In the case of TV Series, it is the series' start year.	Discrete. Interval.
numVotes	Number of votes the title has received.	Discrete. Ratio-scaled. Log-transformed.
numRegions	The regions number for this version of the title.	Discrete. Ratio-scaled. Log-transformed.
totalImages	Total number of images for the title within the IMDb title page.	Discrete. Ratio-scaled. Log-transformed.
totalCredits	Total number of credits for the title.	Discrete. Ratio-scaled. Log-transformed.
criticReviewsRatio	The proportion of reviews from critics on the total number of reviews of a record. For unreviewed records, it is set to 0.	Continuous. Ratio-scaled.
numGenres	The number of genres listed for the variable *genres*.	Discrete. Ratio-scaled.
companiesNumber	Total number of companies that worked for the title.	Discrete. Ratio-scaled.
externalLinks	Total number of external links the title has within the IMDb page.	Discrete. Ratio-scaled.
writerCredits	Total number of writer credits of the title.	Discrete. Ratio-scaled.
averageRating	Weighted average of all the individual user ratings.	Continuous. Ratio-scaled.

Table 3: Numeric attributes after data preparation.

Feature	Description	Type
originalTitle	Original title, in the original language.	Categorical; mostly unique classes (i.e. names).
titleType	The type/format of the title (e.g. movie, short, tvseries, tvepisode, video, etc.).	Categorical, 10 classes.
countryOfOrigin	The country where the title was primarily produced.	Categorical, 153 classes. Some titles can belong to more than 1 class.
genres	The genre(s) associated with the title (e.g., drama, comedy, action).	Categorical, 29 classes. Some titles can belong to more than 1 class.
regions	The regions for this version of the title.	Categorical, 171 unique classes. Some titles can belong to more than 1 class.
rating	IMDB title rating class.	Ordinal, 10 values.
hasAwards	Whether the title has won at least an award	Asymmetric binary.
hasAwardsOrNominations	Whether the title was at least nominated for an award.	Asymmetric binary.
canHaveEpisodes	Whether the title can have episodes.	Asymmetric binary.
hasVideos	Whether the title IMDb title page has at least one video.	Asymmetric binary.
moreCountriesOfOrigin	Whether the title was produced in more than one country.	Asymmetric binary.
hasCriticsReviews	Whether the title has any critic reviews.	Asymmetric binary.
hasUserReviews	Whether the title has any user reviews.	Asymmetric binary.
moreDirectorsCredits	Whether the title has more than one director credits.	Asymmetric binary.
hasQuotes	Whether the title has any quotes within the IMDb page.	Asymmetric binary.

Table 4: Categorical, ordinal, and binary attributes after data preparation.

For reasons that will become apparent in 2.1, we concentrated our analysis on two smaller clusters: C1 (1,668 records) and C2 (3,923). These clusters are later merged in the same super-cluster, which in turn is last to be merged with the rest of the dataset. These findings pertaining the hierarchical structure of our clustering provide and indication both about the similarity between the two clusters in question and about their separation from the rest of the data.

Both C1 and C2 are made up primarily from movies (with some tv series), and according to their distribution w.r.t. *numVotes*, probably contain blockbuster movies and very popular TV series(see fig 4). In addition to that, almost 57% of records from C1 received awards, a proportion that increases to 80% when considering also nominations. C2 records were also well received, with 45% of them having received or being considered for awards. In addition to that, 87% of records from C1 and 45% of records from C2 have videos on their IMDB page and almost all records from both C1 and C2 have both reviews from users and from critics.

2 ADVANCED DATA PRE-PROCESSING

2.1 Anomaly detection

In this section, we adopted methods from different families in order to detect the top 1% anomalous instances in our dataset. When we do not specify otherwise, we fitted all methods on all numeric features of our dataset after scaling them with a Robust Scaler.

In all cases, we then analyzed how candidate outliers were distributed w.r.t. known features

Figure 4: Hierarchical clustering results on tabular data by runtime and number of votes.

and visualized them on the first two components projected by PCA (Fig. 5). These two components alone explain more than 50% of the variance of the dataset. Inspecting the PCA Eigenvectors, we found out that *TotalImages*, *numRegions*, *numVotes*, *companiesNumber*, *externalLinks* and *totalCredits* contributed in a major way to the first component, while *averageRating* and *startYear* contributed negatively to the second component. Fig. 5a provides much needed perspective on the density of records across the two principal components.

2.1.1 HBOS

When analyzing records deemed anomalous by HBOS, we noticed that almost all of them (90%) were movies and very popular movies, at that. In addition to that 44% of candidate outliers received awards, a proportion that increased to 62% when considering also nominations and 70% of them had at least one video on their IMDB page, while all of them were reviewed by both critics and users. Interestingly, all of these proportions are much higher not only when compared with the general distribution in the dataset (cf table 2), but also w.r.t. observed frequencies in movies. It should also be noted that, while TV episodes make up almost half of the dataset, no TV episode was deemed anomalous by HBOS.

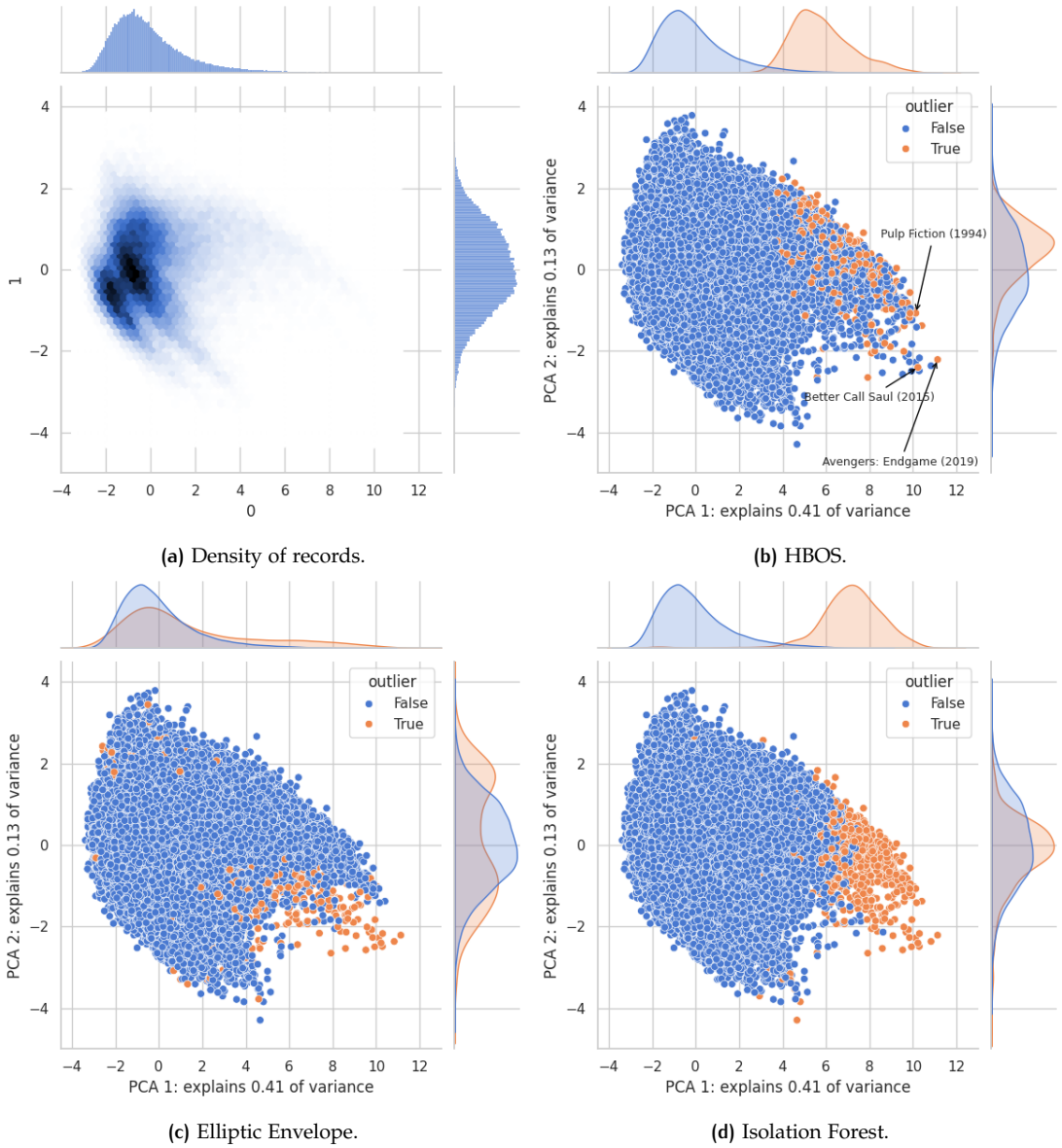
Visualizing HBOS results in latent space, we noticed that outliers and inliers have almost non overlapping distributions with respect to the first component. This is hardly surprising, considering which original features contribute to this component, and merely confirms what we already observed when checking the distribution of inliers and outliers with respect to the original features.

Checking the distribution of candidates against the clustering created in 1.7, we found out that they came almost exclusively from C1 and C2.

HBOS is a very naive approach to anomaly detection on multivariate data that is unable to take into consideration interaction between features. In addition to that HBOS comes with a strong assumption of independence between features; even though we removed the most highly correlated features from our dataset, we still have a number of features with moderate to high positive correlation which are bound to skew the results of this method.

2.1.2 Elliptic Envelope

We adopted Elliptic Envelope as the only depth-based approach that has a python implementation. In contrast to a classic depth-based approach, this method comes with the assumption that the dataset has a Gaussian distribution; since this is not the case for our data, in this case we performed normalization with a Power Transformer in order to reduce skewness and stabilize variance.



(a) Density of records.

(b) HBOS.

(c) Elliptic Envelope.

(d) Isolation Forest.

Figure 5: Results of different methods of anomaly detection.

Candidate outliers extracted with this method show a wholly different distribution w.r.t title type: 261 are TV episodes, 423 tv series and 285 are shorts. The latter we found particularly interesting, since shorts make up only around 10% of records in our dataset, but 35% of candidate outliers.

We were also able to see that outliers are distinct from inliers by runtime (they are much shorter) and year (they are older). The distribution w.r.t. number of credits is also interesting: outliers show a bimodal distribution overlapping with the tails of the inliers distribution, meaning that outliers have exceptionally small or exceptionally big productions.

2.1.3 Isolation Forest

Isolation Forest obtained results that are very similar to those we already observed for HBOS. As a matter of fact, while HBOS and Elliptic Envelope candidate outliers showed minimal overlap, 40% of Isolation Forest candidate outliers were already flagged by HBOS.

Candidate outliers display extreme values on all features associated with big and popular productions. This is also evident by visualizing results on latent features: fig. 5d shows that there is almost no overlap between outliers and inliers along the first component. 96% of candidate outliers come from either C1 or C2, with 80% from C1 alone. Conversely, more than half of the records from C1 are considered anomalous. When confronting fig.

5d with fig. 5a, we can see that while candidate outliers cluster together (at least according to our hierarchical clustering) and thus, logically speaking, they can't be considered isolated, records are extremely sparse on high values of the first component, thus explaining Isolation Forest results.

2.1.4 Conclusions

All methods were united in deeming records representing big and popular productions as anomalous. We found no indication that the anomalous values encountered were in any way erroneous or the result of a data quality problem, so it doesn't make sense to treat them as missing values. While it makes sense that, among all forms of visual entertainment recorded in our dataset, blockbusters would be exceedingly rare, we do not find it appropriate to remove them from the data. We maintain that any analysis of this dataset excluding such titles would fail to capture a very important aspect of the phenomena it claimed to model and would be, as a consequence meaningless.

2.2 Imbalanced Learning

In this section we experimented with several imbalance learning techniques in the context of binary classification. Our goal was to develop a system capable of predicting which titles have received any awards (as already seen in 2, only about 8% our training records did). We can imagine that a similar system could be of interest in a number of contexts, like for recommendation and showcasing purposes, or as an aid to decision-making in the context of licensing agreements, when information about award worthiness isn't readily available (for example in the case of newly released titles).

In our evaluation of different techniques, we decided to relay on the F1 score of the positive class, since we figured that from a business perspective it was equally important detecting as many award worthy titles as possible and avoiding false positives.

2.2.1 Resampling techniques

We established a performance baseline using a KNN classifier with no resampling techniques. We then experimented with three over-sampling strategies (Random Over-sampling, SMOTE, ADASYN), two under-sampling strategies (Random Under-sampling and Edited Nearest Neighbors) and a combination of both (SMOTEEN). In all cases we resampled the dataset as to obtain a perfect balance between the positive and the negative class.

Method	precision	recall	f1
No strategy	.47 ± .02	.31 ± .01	.37 ± .01
Random Over-sampl.	.29 ± .00	.62 ± .01	.40 ± .01
SMOTE	.29 ± .00	.65 ± .01	.40 ± .01
ADASYN	.30 ± .00	.59 ± .01	.40 ± .00
Random Under-sampl.	.24 ± .00	.86 ± .01	.37 ± .00
Edited NN	.50 ± .01	.45 ± .01	.47 ± .01
SMOTE + Edited NN	.30 ± .01	.68 ± .01	.41 ± .01
One Class SVM	.10 ∓ .00	.62 ± .01	.17 ± .00

Table 5: Results of tested imbalanced learning methods on 5-fold cross validation.

For each technique we fine tuned the hyperparameters of the classifier searching for the best F1 score: initially we performed random search on *k*s between 1 and 100, with euclidean and manhattan distance, both weighting neighbors and not. Since we noticed generally better performance when weighting exemplars, we removed unweighted models from the grid search.

We also noticed that for over-sampling methods performance was rapidly decreasing as k got bigger, so we restricted our grid search to odd ks between 1 and 21. Since performance of under-sampling techniques wasn't degrading so rapidly, we extended the number of neighbors considered for these approaches to odd numbers between 1 and 41, but we noticed that scores were plateauing around 10, so we didn't consider it necessary to widen the search beyond that.

We then compared performance of different resampling strategies checking precision, recall and F1 score on 5-fold cross validation (cf fig 5).

Precious insights were also gathered by the inspection of the precision-recall curves (reported in fig 6) and average precision (area under the PR curve).

Over-sampling methods and the combination of SMOTE and Edited NN all show worse ranking capabilities than no resampling strategy at all. In particular these methods demonstrate much worse performance in the early retrieval range of operating points, meaning that they have much more false positives in the top ranked instances when compared with other methods. This is also reflected in the lower precision scores that are not balanced by the higher recall. In addition to that, over-sampling techniques, even when relatively inexpensive at fitting time (Random Over-sampling), are slower at prediction time because of the necessity to compute distances with all records in the over-sampled dataset, and have considerably higher space requirements.

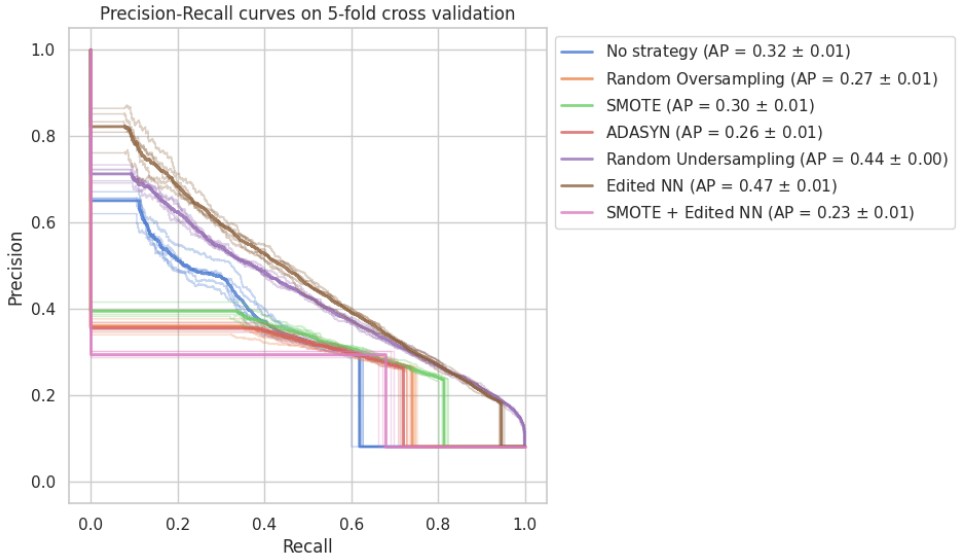


Figure 6: Precision-Recall curves on 5-fold cross-validation for different resampling methods

Performance obtained with Random Under-sampling has similar F1 score to no resampling strategy, but shows better ranking capabilities, as demonstrated both by AP score and inspection of the PR curve. Deceptively low scores obtained by this method in the face of a good PR curve alerted us to the necessity of tuning the decision threshold of our model before deployment.

The best results are obtained with Edited Nearest Neighbor. The PR curve for ENN almost always dominates random under-sampling with minimal crossing-over for higher values of recall.

However, it seems to us that Random Under-sampling may be the preferred choice in some context in which fitting time would be a more pressing concern than the precision of the prediction. In light of this observation, we decided to assess both Edited NN and Random Under-sampling on the test set.

2.2.2 *Tuning of decision threshold and assessment*

Edited Nearest Neighbor already showed satisfactory performance, improving by 0.1 the F1 score of the vanilla KNN and having the best precision among all tested methods (0.50) and recall at least better than the vanilla KNN. Nevertheless, we decided to post-tune the decision threshold of both our best performing models before assessment on the test set.

For the pipeline using Random Under-sampling this resulted in setting a higher threshold (0.77), while for Edited NN the threshold had to be lowered to 0.38.

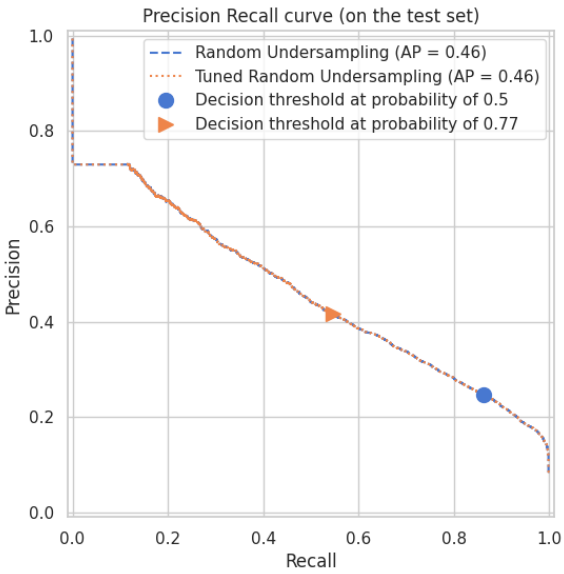


Figure 7: Decision threshold for KNN with Random Under-sampling before and after tuning.

Fig 7 illustrates the effect of changing the decision threshold for the pipeline using Random Under-sampling: we can observe that if the model were to use the default threshold, it would have better recall but much worse precision than when employing the tuned threshold.

We finally assessed both models on the test set, obtaining very similar results: ENN obtained a F1 score of 0.49, while Random Under-sampling 0.47; inspecting the other metrics we can confirm that the worse performance of Random Under-sampling is entirely due to a lower precision (0.42

vs 0.44 for ENN), as we had expected from the inspection of the PR curves in validation.

2.2.3 Exploration of alternative techniques

As we detailed in 2.1, records that received awards and nominations were often deemed anomalous. We decided to leverage this observation adopting a classification based anomaly detection technique for our classification task.

We trained a One Class SVM with RBF kernel without hyperparameter tuning on records that didn't receive any awards in order to learn a discriminative boundary around normal instances. Table 5 show results obtained by this method on 5-fold cross validation. We ultimately deemed the performance of this method unsatisfactory because it produced too many false positives.

3 ADVANCED CLASSIFICATION

In this section, we will examine the performance of various advanced classification methods w.r.t two multiclass classification tasks, aiming to predict the *rating* class and the *titleType* of our records.

Since no features showed high correlation with *averageRating*, while some did exhibit different distribution tendencies w.r.t. *titleType*, we anticipated that the second classification task would yield better results.

In evaluating and comparing our classification models, we selected macro F1 as the main performance metric, given that the different target classes have variable support. For each model, we additionally plotted the Precision-Recall (PR) curves for each class using the one-vs-rest strategy and reported the overall PR AUC. We favored PR curves over ROC curves because they are more informative in multiclass and imbalanced settings.

3.1 Pre-processing

For each model, we built a pipeline which proceeded to first preprocess the data (we used the training and test set, cleaned and prepared as detailed in Sec. 1) and then pass it to the model, in order for it to be fitted on the training set or applied on the test set.

The preprocessing step included scaling the numerical features, if required by the method, and one-hot encoding the categorical variables. For both tasks, we of course dropped the target variable, including *averageRating* for the rating task.

For *countryOfOrigin* and *region*, we only kept the 25 most frequent countries, aggregating the rest into an *Other* category, to reduce dimensionality. For the MultiLayerPerceptron Classifier (and, later, for the regression task), we further reduced dimensionality, keeping only the top nine countries for both features and dropping *hasAwards* on the account of its positive correlation to *hasAwardsOrNominations*, in order to further reduce computational costs and training time during hyperparameter tuning.

3.2 Logistic Regression

To set a baseline, we first tried a Logistic Regression model. As the algorithm assumes that the features follow a Gaussian distribution, we scaled numerical features using PowerTransformer, which reduces skewness and stabilizes variance.

3.2.1 Rating

Using the default parameters yielded a macro F1 of 0.144 and an accuracy of 0.388. We performed a grid search using 5-fold cross validation, which resulted in a very small increase, with a 0.146 average macro F1 on the validation sets and 0.152 on the training set, indicating no overfitting. We tested the following hyperparameters (the values for the final model are in bold):

```
{ 'penalty': ['l2', 'l1', 'elasticnet'],
  'C': [0.001, 0.01, 0.1, 1, 10, 100],
  'l1_ratio': [0.1, 0.3, 0.5, 0.7, 0.9],
  'solver': ['lbfgs', 'sag', 'saga', 'newton-cg',
      'newton-cholesky', 'liblinear']  }
```

On the test set, the final model achieved a **macro F1** of **0.145** and an **accuracy** of **0.388**, comparable to the default model. The poor performances across decision thresholds are confirmed by the **PR AUC** of only **0.193**.

The model has managed to generalize some information only for the classes with the highest support: the performances peak for the mode, *(7,8]* (F1: 0.544), its neighboring classes are confused for it most of the time, and as we move to the tail of the bell distribution, it achieves consistently poor results (F1 < 0.05).

Looking at the model's coefficients, the overall most important feature is *hasCriticReviews* (mean absolute value > 1.2, while for the other coefficients in the top 15 it is in the range [0.6, 0.7]): it is the first or second-biggest coefficient for the best performing classes (i.e. for the interval *(5,8]*),

and for all of them it takes on a negative value, suggesting that having critical reviews lowers the probability of a record of having an 'average-to-high' rating.

3.2.2 Title Type

For this task, the final model resulting from the grid search achieved an average validation macro F1 of 0.653 (on the training set: 0.664). The final parameters chosen were: `'C': 100, 'penalty': 'l2', 'solver': 'newton-cg'`.

On the test set, the model achieved a **macro F1** of **0.649** and an accuracy of **0.908**; the improvement w.r.t. the default parameters (0.647 and 0.908) is, again, quite small. The **PR AUC** is **0.704**; the curve for each class reflects its performance.

Given that both tasks have 10 target classes, our intuition about *titleType* yielding better results is confirmed.

The four most popular classes (*tvEpisode*, *short*, *movie*, *tvSeries*) achieve great results, with F1 > 0.9. The best performing and most popular class, *tvEpisode*, peaks with 0.978. All other classes (except *tvShort*) achieve a F1 score > 0.4, which is still significantly more than random chance. *Videogame*, in spite of its low support, manages to achieve very good results (F1 0.76).

The 15 most important features are reported in Fig. 8. Overall, the two most informative features are:

- *canHaveEpisodes*: it has very high positive values for *tvSeries* (+13.15) and *tvMiniSeries* (+12.18) and negative for all other classes; it's also the most important feature for *movie* (-4.75). This means that the model has learned to use this feature to distinguish serialized from non-serialized content.

- genre *Short*: of course, it's the most important feature for *short* and *tvShort*, and it's the second most important for *video* with positive values; it's high up in the ranking with a negative value for *movie*, *tvEpisode*, and *tvMovie*.

3.3 Linear Support Vector Machine

For Support Vector Machines (SVMs), we scaled numerical features using RobustScaler, as our data contains outliers. We tested the linear kernel and the Radial Basis Function kernel separately to exploit sklearn's optimized LinearSVC implementation.

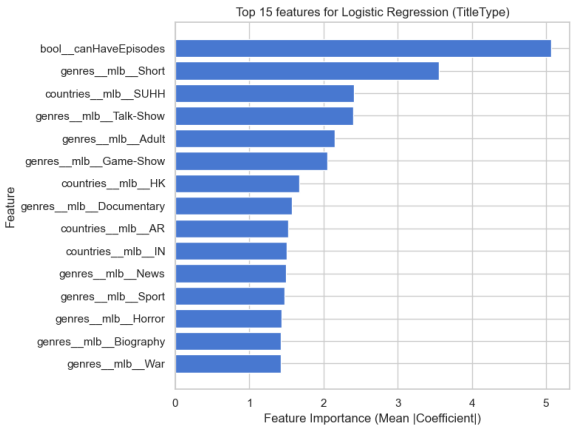


Figure 8: Top 15 most important features for *titleType* (LR).

3.3.1 Rating

For LinearSVC, we tested the following parameter grid using a grid search with 5-fold cross validation:

```
{ 'C': [0.001, 0.01, 0.1, 1, 10, 100],
  'penalty': ['l1', 'l2'],
  'loss': ['squared_hinge', 'hinge'],
  'max_iter': [5000],
  'class_weight': [None, 'balanced']  }
```

For *rating*, it resulted in an average macro F1 of 0.158 for validation and of 0.164 on the training set.

On the test set, w.r.t. Logistic Regression, the model achieved a higher **macro F1** (**0.158**), but a lower **accuracy** (**0.270**). Indeed, the F1 for the best performing classes has decreased (*(7,8]*: 0.496 vs 0.544; *(6,7]*: 0.215 vs 0.356, *(5,6]*: 0.259 vs 0.283), in favor of slightly better performances for classes with lower support: the decrease in accuracy is linked to the fact that the model tries to predict these lower support classes more often, rather than focusing merely on the central classes. This led to a notable increase in F1 for *(3,4]* (0.113 vs 0.04) and *(9,10]* (0.145 vs 0.046), but it is clear that the model still struggles. The **PR AUC** is also slightly worse, **0.175**.

The confusion matrix confirms this tendency: if for Logistic Regression predictions were concentrated on the best performing classes, with many zeros populating the lowest classes, now confusion is spread throughout.

The most important feature is *tvEpisode* (mean absolute value: 0.175): it is at number one, with negative values, for classes from *(2,3]* to *(5,6]* — this would suggest that serialized content is less likely to have bad rating. Classes *(7,8]* and *(8,9]* both have *movie* as most important feature, with negative contributions (-0.207 and -0.190, respec-

tively). This might explain why the latter is mostly classified as the former, which is the class with the highest support.

3.3.2 Title Type

The model returned by the grid search yielded an average validation macro F1 of 0.646 (vs 0.654 on the training set); the parameters were `'C': 0.1, 'class_weight': 'balanced', 'loss': 'squared_hinge', 'max_iter': 5000, 'penalty': 'l1'`.

On the test set, the results were slightly lower than Logistic Regression: **0.640 macro F1**, **0.889 accuracy**, **0.674 PR AUC**. Indeed, performances for almost all classes have decreased to some degree, with a remarkable dip in precision for *videogame* (0.486 vs 0.799).

The overall most important features are the same (*canHaveEpisodes* and genre *Short*), but now only the first stands out (mean absolute value close to 2.5), while *Short* has a similar value to the rest of the top features, < 1.

Unexpectedly, the most important feature for the *titleType short* is now *canHaveEpisodes*, with quite an 'advantage' (coef: -2.18, vs genre *Short* 1.43).

3.4 Support Vector Machine with Radial Basis Function (RBF)

The power of SVMs comes at a high computational cost, which is why we only tested one kernel function (RBF) and why we opted for an 80/20 holdout, instead of a k-fold cross-validation, to carry out hyperparameter tuning. We also defined a small hyperparameter grid:

```
{ 'C': [0.1, 1, 10],
  'gamma': ['scale', 0.01, 0.1],
  'class_weight': [None, 'balanced']  }
```

3.4.1 Rating

The best model returned by the grid search had the following hyperparameters: `'C': 10, 'gamma': 0.1, 'class_weight': None`. It achieved a macro F1 of 0.265 on the validation set and of 0.698 on the training set, suggesting overfitting. We therefore reran the grid search, excluding the highest values for C and gamma in order to increase regularization. The best performing model was less overfitted, but also achieved lower results (macro F1 0.206 for validation and 0.316 for training). Its parameters are: `'C': 1, 'class_weight': 'balanced', 'gamma': 'scale'`.

On the test set, the overfitted model proved to be the best one, achieving a **macro F1** of **0.257** (vs 0.219) and an **accuracy** of **0.431** (vs 0.314). The resulting **PR AUC** is **0.231**. This model is also the best performing one thus far, with improvements observed across almost all classes.

Looking at the confusion matrix, we can see that the model doesn't completely snub lower support classes, like Logistic Regression did, but is more accurate than LinearSVC. Especially for the more popular (and better performing) classes, confusion is mostly among neighboring classes.

3.4.2 Title Type

Similarly to what had happened with *rating*, the first grid search returned an overfitted model (macro F1 for validation: 0.746, vs for training: 0.905). Forcing regularization resulted in less overfitting, but also in a much poorer performance (validation macro F1: 0.206, train macro F1: 0.316); for this reason, we chose the first classifier. Its parameters are: `'C': 10, 'class_weight': None, 'gamma': 'scale'`.

On the test set, this model clears all previous attempts, with a **macro F1** of **0.746** and an **accuracy** of **0.938**.

The performances for all classes increased. *tvEpisode* now achieves an F1 of 0.99; *videogame* improved greatly, reaching 0.884 F1. Nevertheless, we only get a **PR AUC** of **0.689**; indeed, the curves in Fig. 9 show how the precision for half of the classes sinks at around 60% recall. This means that, while the model shows excellent performance metrics on a per-class basis, it has difficulty maintaining a reliable performance when attempting to comprehensively identify instances of minority classes.

3.5 Random Forest

Since this method is based on trees, no scaling is needed for numerical features. To cut computational costs, we used 3-fold cross validation during hyperparameter tuning.

Indeed, there are many possible hyperparameters to tune. For each task, we ran three Randomized Searches, for a varying number of estimators — 100, 300 and 500, respectively. For each, we tested the same 100 random combinations of parameters from the following grid:

```
{ 'criterion': ["gini", "entropy"],
  'max_depth': [None, 5, 15, 25],
  'min_samples_split': [2, 5, 10],
  'min_samples_leaf': [1, 5, 10],
  'class_weight': [None, 'balanced']  }
```

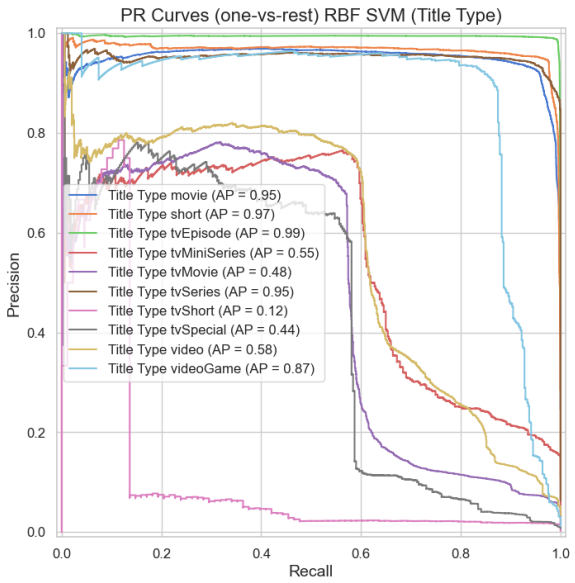


Figure 9: PR curves for *titleType* (SVM with RBF).

3.5.1 *Rating*

Before proceeding with hyperparameter tuning, we first tested a Random Forest using default parameters, which yielded an average validation macro F1 of 0.296, and an even better 0.302 macro F1 score on the test set.

The analysis of the results from the Randomized Searches revealed that the number of estimators is not a strong driver of performance: the average validation macro F1 doesn't change significantly among hyperparameter configurations differing only in the number of trees. Although the performance of the other two models was very close, we chose the best model with 300 base classifiers, which yielded an average macro F1 equal to 0.308 for validation, and equal to 0.968 for train. Its parameters are: `'n_estimators': 300, 'min_samples_split': 5, 'min_samples_leaf': 1, 'max_depth': None, 'criterion': 'gini', 'class_weight': 'balanced'`.

Whether such a gap between validation and train macro F1 should be interpreted as a sign of overfitting for Random Forest is debated in the literature: it has been argued that, even if the single trees can overfit, they tend to do so in different ways, so that combining them in an ensemble reduces the overall variance. Thus, although pruning the individual trees can be beneficial, unless the dataset is very simple, we could consider Random Forest as particularly robust against overfitting.

The model's performance on the test set increases, making it the best model thus far: it achieves a **macro F1** of **0.325**, an **accuracy** of **0.452**, and a **PR AUC** of **0.318**.

It achieves the most balanced performance, with six classes out of ten scoring an F1 >= 0.3. Confusion remains spread out for the minority classes (from *0* to *4*), whereas for the others it remains concentrated amongst neighboring classes.

The Mean Decrease in Impurity (MDI) returned as the most important features *numVotes* and *startYear* (almost 0.10), followed by *totalCredits* (around 0.9).

3.5.2 *Title Type*

The default model achieved an average macro F1 of 0.707 on validation and a score of 0.728 on the test set.

The performances for the different parameters combinations tested during the Random Searches were again very similar, with the configurations using 100 estimators yielding marginally lower scores. Indeed, the three best models shared the same hyperparameter values. We chose the configuration with 300 trees: the validation performances were only marginally better than that of the 500-trees model, but we also favored simplicity.

The model yielded 0.767 and 0.975 for validation and training, respectively. Its hyperparameter configuration is the following: `'n_estimators': 300, 'min_samples_split': 10, 'min_samples_leaf': 1, 'max_depth': None, 'criterion': 'entropy', 'class_weight': 'balanced'`.

On the test set, it clears previous models, achieving **0.777 macro F1** and **0.940 accuracy**. Half of the classes yield F1 > 0.95. It generalizes better on the minority classes *tvMovie* and *tvMiniSeries*: although there is still confusion with the more popular *movie* and *tvSeries*, they are correctly classified most of the time (cf. the confusion matrix in Fig. 11).

This model's superiority across decision thresholds is supported by the PR curves in Fig. 10, and further quantified by its **PR AUC** of **0.818**.

Checking the 15 most important features (Fig 12), we see how the two most important features for the Logistic Regression model, *canHaveEpisodes* and the genre *Short*, have been overtaken by *runtimeMinutes*. Indeed, we had uncovered a relationship between *runtimeMinutes* and *titleType* in Subsec. 1.6.

Finally, it's worth pointing out that prior experimentation keeping the default hyperparameter `'class_weight': None` instead of `'balanced'` lead to a best model scoring a significantly lower

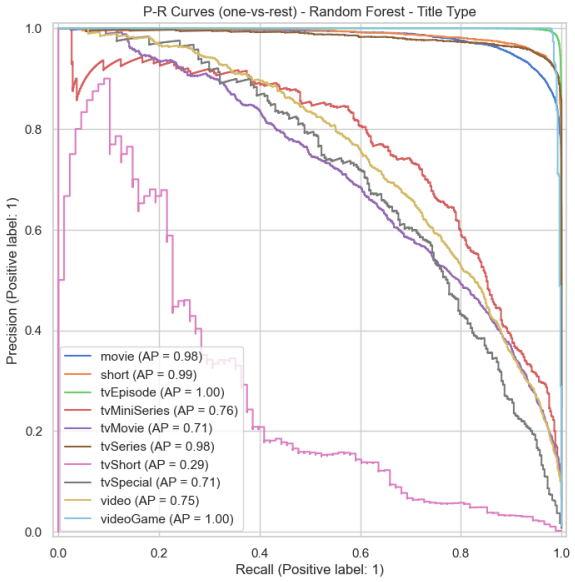


Figure 10: PR curves for *titleType* (RF).

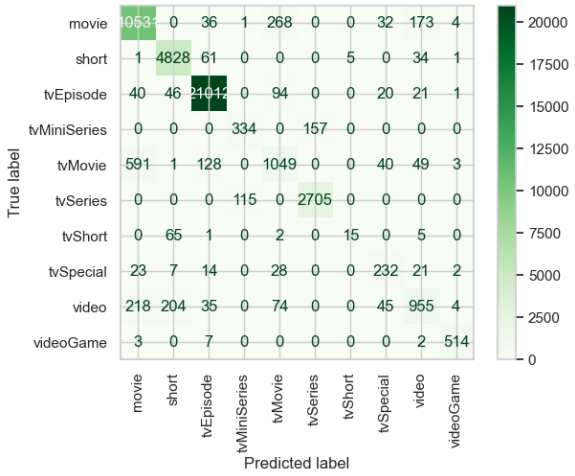


Figure 11: Confusion matrix for *titleType* (RF).

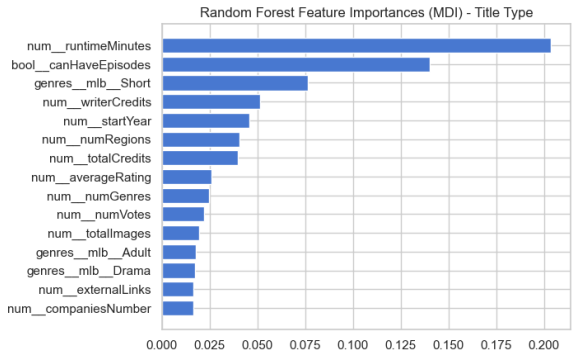


Figure 12: Top 15 most important features for *titleType* (RF).

macro F1 score of 0.728. This is indicative of the importance of this hyperparameter for this unbalanced task.

3.6 LightGBM

Within the Gradient Boosting family of algorithms, we selected LightGBM for its efficiency. Since it exploits Decision Trees as base classifiers, numerical features don't require scaling.

Similarly to what we did for Random Forests, we performed hyperparameter using a Randomized Search with 100 iterations and a 3-fold cross-validation to test the same random combinations with 100 and 300 estimators (we excluded 500 estimators because it resulted too computationally expensive). The hyperparameter configurations were sampled from the following grid:

```
{ 'boosting_type': ['gbdt', 'goss', 'dart'],
  'num_leaves': np.arange(20, 100, 10),
  'max_depth': [-1] + list(np.arange(3, 13)),
  'learning_rate': [0.01, 0.03, 0.05, 0.1],
  'reg_alpha': [0, 0.01, 0.1, 1],
  'reg_lambda': [0, 0.01, 0.1, 1] }
```

3.6.1 *Rating*

The best resulting model has the following hyperparameters: `'n_estimators': 300, 'reg_lambda': 0.1, 'reg_alpha': 0.01, 'num_leaves': 50, 'max_depth': 10, 'learning_rate': 0.1, 'boosting_type': 'gbdt'`. It yielded an average cross-validation macro F1 of 0.285 and of 0.819 on the training set, showing signs of overfitting.

Given the same hyperparameter configuration, increasing the number of estimators leads to a better performance, but also to a larger overfitting gap (i.e. the difference between training and validation macro F1).

Fig. 13 shows how the overfitting gap grows exponentially as performances increase, suggesting that the data is hard to model.

On the test set, the model achieved **0.289 macro F1**, **0.440 accuracy**, and **0.299 PR AUC**, positioning itself at second place, under Random Forest.

Consistently with Random Forest, *startYear* and *numVotes* emerged as the most important features, with comparable gain values.

3.6.2 *Title Type*

Analyzing the results of the Random Searches leads to the same discovery we had for *rating*: given a certain hyperparameter configuration, the average cross-validation macro F1 grows for 300

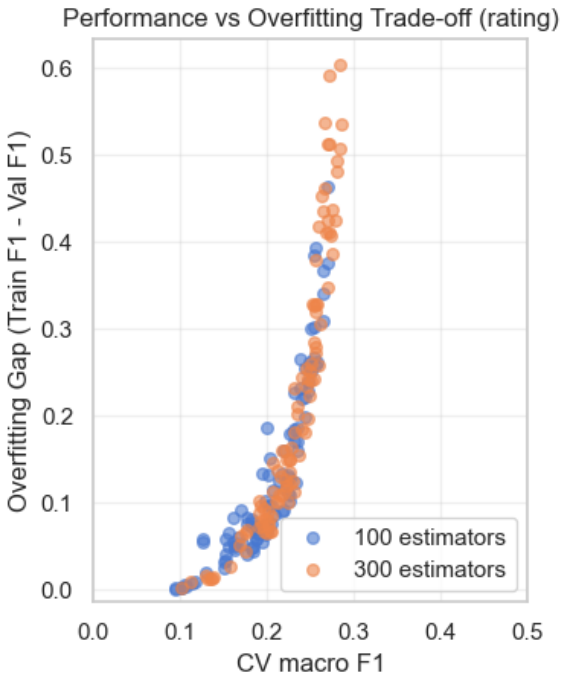


Figure 13: Trade-off between avg validation macro F1 and overfitting gap for *rating* (LightGBM).

estimators w.r.t. 100, but so does the overfitting gap. Nevertheless, Fig. 14 illustrates how the relationship between the overfitting gap and the validation macro F1 grows more slowly. Moreover, the best performing models are only moderately overfitted (gap of around 0.2 vs 0.6 for *rating*.)

The best model yields a cross-validation macro F1 of 0.791 against a training macro F1 of 0.982. Its parameters are: `'n_estimators': 300, 'reg_lambda': 1, 'reg_alpha': 0, 'num_leaves': 20, 'max_depth': 10, 'learning_rate': 0.1, 'boosting_type': 'gbdt'`.

On the test set, it reached **0.795 macro F1**, **0.949 accuracy**, and **0.847 PR AUC**, surpassing Random Forest. Performances for almost all classes improved, with *tvEpisode* reaching 0.99 F1. The lowest F1 score, excluding *tvShort*, is 0.688 for *tvMovie*. The PR curves in Fig. 15 also highlight the improvement for the minority class *tvSpecial*, which indeed scored 0.747 F1 (vs 0.667 for Random Forest).

Looking at the feature importance (Fig. 16), the top three most important features are the same as for Random forest, except that genre *Short* is slightly more important than *canHaveEpisodes*, and that *runtimeMinutes* by far the most significant.

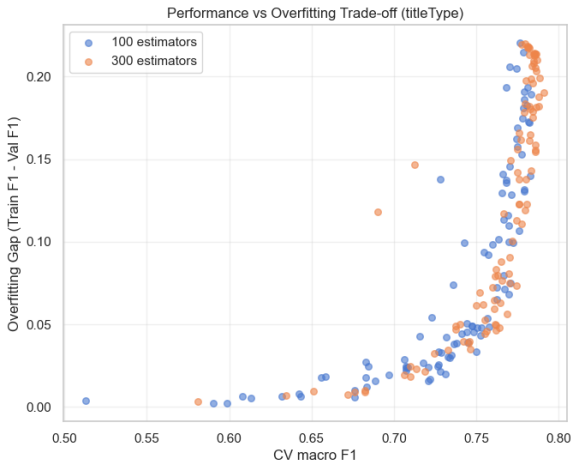


Figure 14: Trade-off between avg validation macro F1 and overfitting gap for *titleType* (LightGBM).

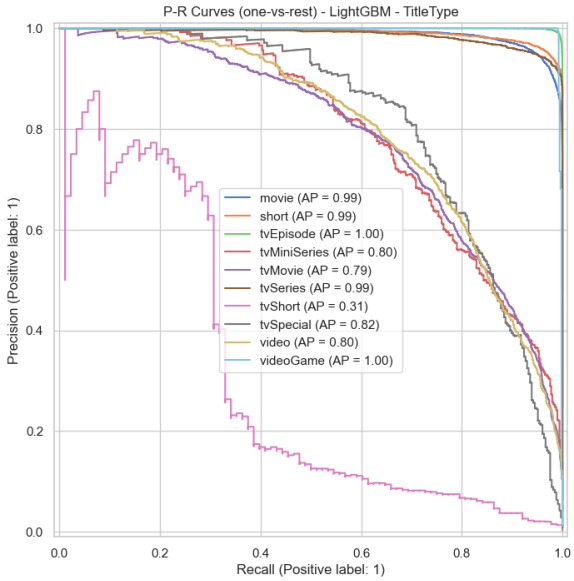


Figure 15: PR curves for *titleType* (LightGBM).

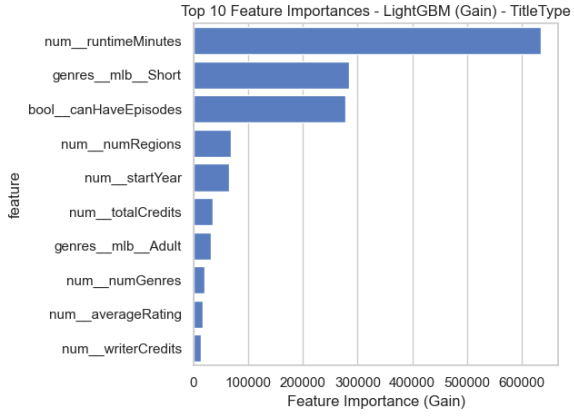


Figure 16: Top 10 most important features for *titleType* (LightGBM).

3.7 Multilayer Perceptron

As anticipated in Subsec. 3.1, for this model, we reduced the dimensionality of the dataset to make training faster. We also scaled numerical features using RobustScaler.

Since a comprehensive grid search for hyperparameter tuning would have been too computationally expensive, we ran three Randomized Searches using 80/20 holdout and 100 iterations for a varying number of hidden layers, ranging from one to three. As for the size of hidden layers, we tested the following architectures, varying the number of hidden units and the decreasing patterns:

```
[(32,), (64,), (128,)],
[(64,32), (128,64), (128,32)],
[(128,64,32), (64,64,32)]
```

The rest of the parameters are sampled from the following grid:

```
{ 'activation': ['relu', 'tanh'],
  'alpha': [0.0001, 0.001, 0.01],
  'learning_rate_init': [0.0001, 0.001, 0.01] }
```

For both tasks, the MLPClassifier positioned itself in the lower half of the ranking. Further exploration of more advanced Deep Learning architectures is needed to better leverage the potential of Neural Networks.

3.7.1 *Rating*

We first tried using the default parameters, which yielded 0.239 macro F1 on the validation set and 0.316 on the training set, whereas on the test set it achieved 0.240 macro F1 and 0.415 accuracy.

After hyperparameter tuning, the best model ended up having two hidden layers, yielding 0.262 validation macro F1 and 0.387 for training. Its parameters are: `'hidden_layer_sizes': (128, 64), 'learning_rate_init': 0.001, 'alpha': 0.01, 'activation': 'relu'`.

Plotting the trade-off between the overfitting gap and the validation macro F1 (Fig. 17) shows that the gap tends to increase as performance increases, but the relationship is not as clear as it was for LightGBM. Moreover, most underfitted models only have one layer.

On the test set, the model achieved a **macro F1** of **0.244**, an **accuracy** of **0.413** (lower than the default model), and a **PR AUC** of **0.244**.

We also tried implementing early stopping, which stopped the training at around 40 epochs (vs 200): the macro F1 sank to 0.236, but the accuracy increased slightly, reaching 0.425.

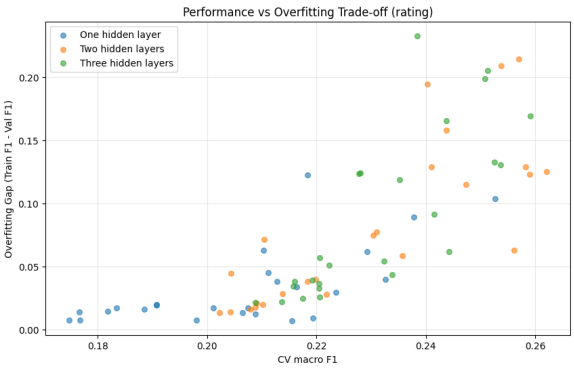


Figure 17: Trade-off between validation macro F1 and overfitting gap for *rating* (MLP).

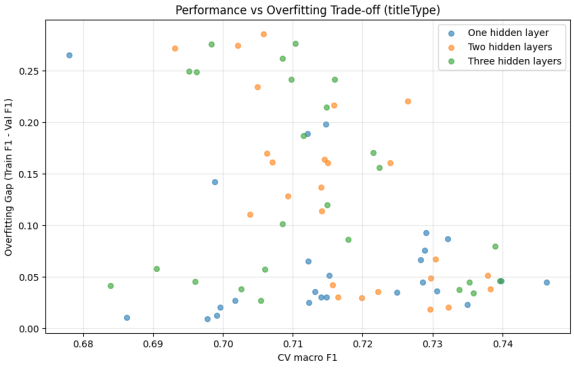


Figure 18: Trade-off between validation macro F1 and overfitting gap for *titleType* (MLP).

3.7.2 *TitleType*

In this case, in terms of macro F1 the default model reaches 0.729 on the validation set, 0.857 on the training set, and 0.730 on the test set. It also achieves 0.931 test accuracy.

After hyperparameter tuning, the best model turns out to be relatively simple, with just one layer and 64 hidden units. It yields a macro F1 of 0.746 on validation and 0.791 on the training set, excluding overfitting. Its hyperparameters are: `'alpha': 0.01, 'hidden_layer_sizes': (64,), 'learning_rate_init': 0.001, 'activation': 'tanh'`.

Indeed, the scatterplot in Fig. 18 reveals how more layers do not necessarily lead to a better performance, but they do lead to a tendency to overfit.

On the test set, the model still achieves **0.738 macro F1**, coupled with **0.934 accuracy** and **0.784 PR AUC**. With early stopping, the training is halted at around 130 epochs. It leads to a drop in macro F1 to 0.728, as the model performs slightly worse on minority classes.

Model	Macro F1	Accuracy	PR AUC
RF	**0.325**	**0.452**	**0.318**
LightGBM	0.289	0.440	0.299
SVM w/ RBF	0.257	0.431	0.231
MLP	0.244	0.413	0.244
LinearSVM	0.158	0.270	0.175
LR	0.145	0.388	0.193

Table 6: Performance comparison of different models on *rating*.

Model	Macro F1	Accuracy	PR AUC
LightGBM	**0.795**	**0.949**	**0.847**
RF	0.777	0.940	0.818
SVM w/ RBF	0.746	0.938	0.689
MLP	0.738	0.934	0.784
LR	0.649	0.908	0.704
LinearSVM	0.640	0.889	0.674

Table 7: Performance comparison of different models on *titleType*.

3.8 Conclusions

The performances of the different models for the two tasks are summarized in Table 6 and 7. Ensemble methods have clearly outperformed the rest; the comparatively poorer performance of Logistic Regression and Linear SVC indicates that linear models are insufficient to capture the underlying class boundaries.

As anticipated, the task of predicting *titleType* yielded better results across models, which, just like for DM1, is likely due to better class separability and stronger feature association.

All models achieved great results for the more popular classes, with an F1 around 0.9, while the results for the minority classes varied from around 0.4 to a robust 0.7 for the best models. In particular, the best models achieved great results for *videogame*, and were better at separating *tvMovie* and *tvMiniSeries* from their more popular counterparts, *movie* and *tvSeries*.

With regard to *rating*, the performance of every model reflects the left-skewed class distribution: performances peak for class *(7,8]*, the mode, and quickly degrade. The models do not have a chance to generalize on the lowest and the highest classes, especially those in the 'tail', due to lack of representation. The best models, however, managed to concentrate their confusion on neighboring classes, especially around the peak of the distribution. Future work could explore undersampling and oversampling techniques, which might help improve generalization.

Finally, some challenges related to overfitting were observed, despite using hyperparameter grids designed to explore different trade-offs between bias and variance; investigating stronger regularization strategies or data balancing techniques may help improve robustness.

4 EXPLAINABILITY: SHAP

In this section, we'll expand the global overview on feature importance returned by the Random Forest model with local explanations of meaningful records returned by SHAP. In particular, we used Tree SHAP, as it is designed for ensembles of trees. Since it produced more meaningful results, we focused on the *titleType* task.

Initially, our plan was to compute SHAP values for the entire test set and run K-Means on the explanations, in order to focus our analysis on the centroids or medoids, i.e. the 'prototypical' explanations. However, this approach proved to be too computationally expensive. We therefore decided to only compute the SHAP values for 'prototypical' instances of the test set. In order to do that, we ran K-Means on the test set and we approximated the medoids by selecting the test instances closest to the centroids. For simplicity, we clustered the data using only numerical features, scaled with RobustScaler. We employed the elbow method to determine the optimal value of clusters, which resulted in k = 13.

Twelve out of the 13 instances either belonged to or were classified as one of the top three *titleTypes*: *movie*, *tvEpisode* and *short*. Three instances were misclassified. Overall, we find that *runtimeMinutes* is the most important feature for almost all records, confirming what anticipated by the Random Forest MDI importance (cf. Fig. 12).

All four ***tvEpisodes*** were correctly classified. For all of them, the most important feature was *runtimeMinutes*, except for the only one that was classified with 100% confidence (Medoid n. 5). Here, the biggest contribution (+0.29) has been made by *numRegions* (cf. Fig 19). Looking at the record's values, *numRegions* is set to zero: we assume this lack of information is preponderant among *tvEpisodes* in the dataset, therefore the model has overfitted on it. This also rings true for *regions*: the feature *regions_Other* has a negative contribution for those *tvEpisodes* which do have a country for *regions*, positive otherwise.

Medoids n. 8 and n. 9 are ***shorts***: both have as most important feature the genre *Short* (+0.28 and +0.23, respectively), followed by *runtimeMinutes* (+0.15 and +0.19) and *canHaveEpisodes* (+0.06 and

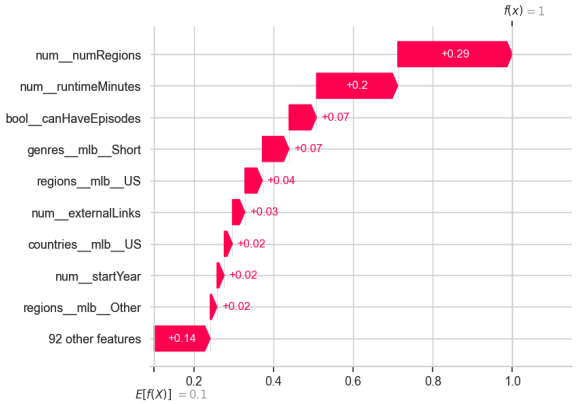


Figure 19: Waterfall plot for Medoid n. 5 (correct *tvEpisode*, 100% confidence).

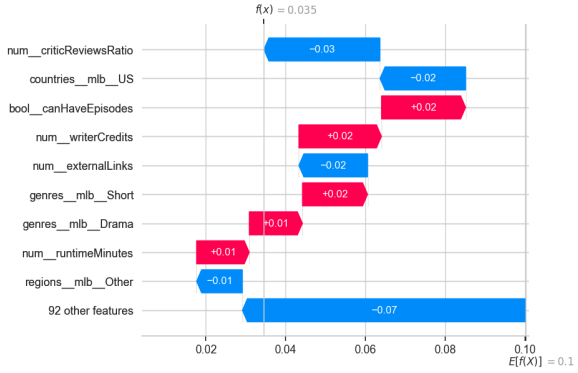


Figure 20: Waterfall plot for medoid n. 11 for its correct label *tvMovie* (3.5% confidence).

+0.04). They were confidently classified, with 97.7% and 87.3% probability.

All four correctly classified ***movies*** have high probability (>80%); their two most important features are *runtimeMinutes*, with a contribution between +0.2 and +0.29, followed by *canHaveEpisodes* (around 0.07; of course all records had as value *False*).

The following instances were misclassified:

- Medoid 1: classified as *tvMovie* (50.9%) instead of *video* (10.4%);
- Medoid 3: classified as movie (83%) instead of *tvMovie* (11.4%);
- Medoid 11: classified as movie (95.2%) instead of *tvMovie* (3.5%);

As for Medoid 3 and 11, the misclassification is quite confident: as we know, this minority class tends to be confused for its more popular counterpart. Their waterfall plot for the class *movie* is similar to the ones discussed above. Looking at the waterfall plots for their real target class, we see that both records are mostly affected by the sum of small negative contributions of the remaining features. Interestingly, both have as main negative feature *criticReviewsRatio* (-0.02 and -0.03): they have relatively high values (50% and 70%), which the model might have learned to be unusual for this type of TV product. Focusing on Medoid 11 (Fig. 20), we can infer that *externalLinks* has an important negative contribution towards *tvMovie* for the same reason. Not as easily interpretable is the negative contribution of the fact that its *countryOfOrigin* is the US, as it is also negative for Medoid 3, which was produced in Germany: there might be an interaction of features at play for this decision.

For Medoid 1, on the other hand, the classification was not as confident. The biggest singular positive contributions towards *tvMovie* were made by *runtimeMinutes* and *writerCredits* (+0.06), though the probability was mostly influenced by the sum of many smaller positive contributions. W.r.t. the true label *video*, the biggest negative contributions are due to the fact that the title was produced and distributed in the US (-0.03 and -0.02), and the fact that it isn't tagged as genre *Adult* (-0.02).

5 ADVANCED REGRESSION

In this section, we will discuss the performance of two advanced regression methods in predicting the *averageRating*, Random Forest and XGBoost.

We used the same preprocessing pipeline detailed in Subsec. 3.1 and the feature set we had used for the Multilayer Perceptron, detailed in Subsec. 3.1. As both methods are based on trees, numerical features did not require scaling.

We evaluated performances using the Determination Coefficient (R2) and the Mean Squared Error (MSE). To set a baseline, we used a DummyRegressor which always predicts the mean of the target values observed in the training set, which yielded a **R2** of **0** and a **MSE** of **1.819** (equal to the variance of the target variable).

5.1 Random Forest Regressor

To tune our model, we used Randomized Search with 3-fold cross-validation and tested 100 random combinations from the following grid for 100, 300 and 500 base models:

```
{ 'n_estimators': [100],
  'max_depth': [None, 5, 15, 25],
```

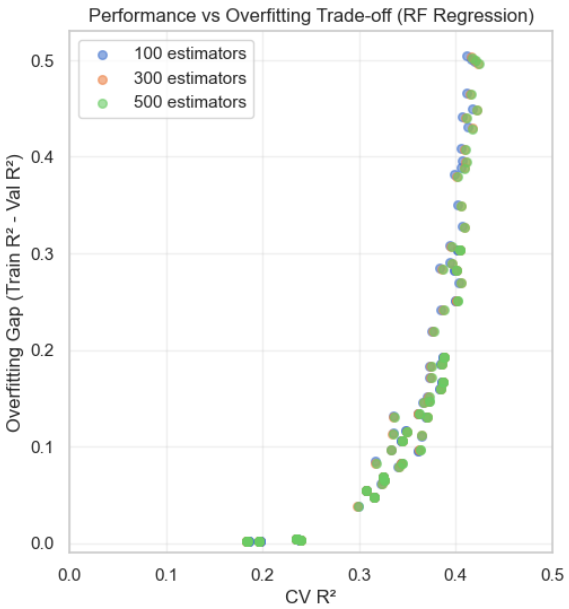


Figure 21: Trade-off between avg validation R2 and overfitting gap for *averageRating* (RF Regressor).

```
'min_samples_split': [2, 5, 10],
'min_samples_leaf': [1, 5, 10],
'max_features': [None, 'log2', 'sqrt', 0.5]  }
```

Comparing the average validation R2 and the overfitting gap among parameter combinations that differ only in the number of estimators, no significant differences emerge. The scatterplot in Fig. 21 illustrates an exponential relationship between the overfitting gap (defined as the difference between the average training and validation R2) and the performance of the regressors (i.e. its average validation R2). This pattern is similar to the one we had observed for classification.

The best model had the following hyperparameters: `'n_estimators': 500, 'min_samples_split': 2, 'min_samples_leaf': 1, 'max_features': 0.5, 'max_depth': None`. Its results were only marginally better than the best 300-trees configuration. It yielded 0.424 average validation R2 (vs 0.921 on the training set) and 1.047 MSE (vs 0.144 on the training set). The same considerations made about classification for Random Forest and overfitting apply for regression.

On the test set, the model achieved an **R2** of **0.443** and an **MSE** of **1.013**, surpassing the baseline. The correlation between real and predicted values is high-to-moderate (Pearson r = 0.667, Spearman ρ = 0.670). As shown in Fig. 22, the model struggles particularly with lower rating values, which correspond to the tails of the target

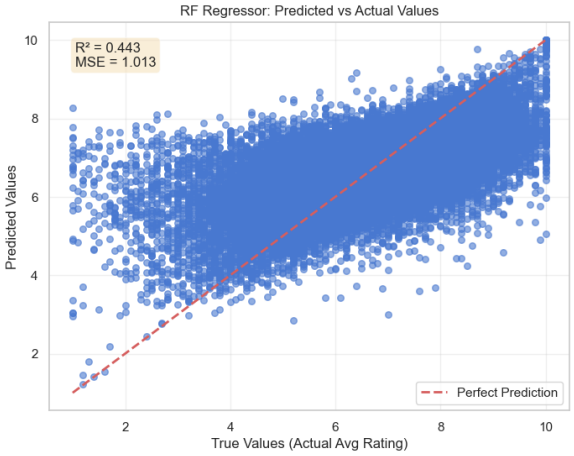


Figure 22: Relationship between real and predicted values of *averageRating* (RF Regressor).

variable's distribution, and does best for values around the mode of the distribution.

5.2 XGBoost Regressor

The grid used for hyperparameter tuning is the following:

```
{ 'max_depth': [3, 6, 9],
  'learning_rate': [0.05, 0.1, 0.3],
  'min_child_weight': [1, 2, 3, 4],
  'gamma': [0, 0.1, 0.5, 1],
  'reg_alpha': [0, 0.01, 0.1],
  'reg_lambda': [1, 1.5, 2, 5]  }
```

Once again, we compared the performance and overfitting gap for the same configurations using a different number of estimator (100, 300 or 500). In this case, more trees led to both better performances and a bigger overfitting gap. Interestingly, plotting the R2 overfitting gap as a function of the average validation R2 seems to suggest the existence of two slightly different exponential trends (Fig. 23).

The best model's parameters are `'n_estimators': 500, 'reg_lambda': 5, 'reg_alpha': 0.01, 'min_child_weight': 2, 'max_depth': 9, 'learning_rate': 0.1, 'gamma': 0`. It yielded 0.397 average validation R2 and 1.097 MSE (vs 0.759 and 0.439, respectively, on the training set).

On the test set, its performance was worse than that of the Random Forest: it achieved **0.409 R2** and **1.075 MSE**. The predictions are only moderately correlated to the true values (Pearson r = 0.640, Spearman ρ = 0.637; cf. Fig. 24).

We also tested a model whose validation metrics represented a compromise in the variance-bias trade-off (average train R2: 0.543,

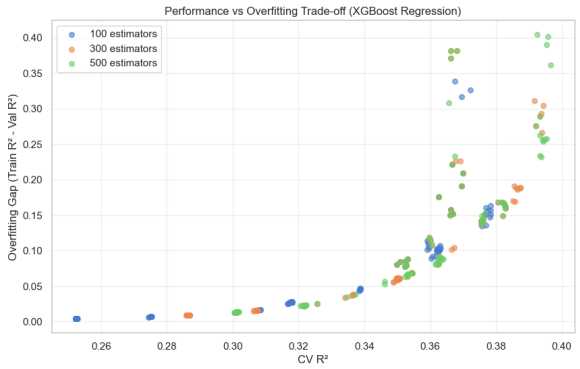


Figure 23: Trade-off between avg validation R2 and overfitting gap for *averageRating* (XGBoost Regressor).

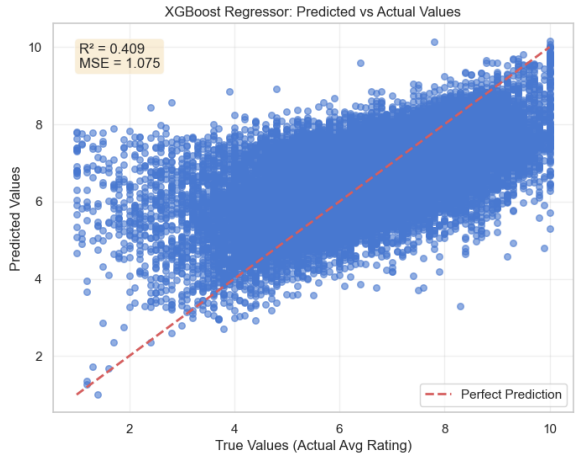


Figure 24: Relationship between real and predicted values of *averageRating* (XGBoost Regressor).

validation: 0.383). Its performances on the test set were slightly worse: 0.391 R2, 1.108 MSE. Its parameters were `'n_estimators': 500, 'reg_lambda': 5, 'reg_alpha': 0.01, 'min_child_weight': 2, 'max_depth': 9, 'learning_rate': 0.1, 'gamma': 0.5`.

6 TIME SERIES ANALYSIS

6.1 Data Understanding and Preparation

This dataset contains data on the domestic (US and Canada) daily gross income at the box office for 1,134 movies, measured from their release date up to 100 days after release. The dataset was extracted from Box Office Mojo[1] and contains no missing values, because shorter time series were padded using noise augmented mean. Each time series was associated with the movie IMDB id, genre and rating (both continuous average rating and a binned version). In addition to the provided data, we used the IMDB free api[2] to retrieve some additional information about the titles represented in the dataset: title and release year of the movie, runtime (in minutes) number of votes, countries of origin, number of awards and nominations. We additionally aggregated the box office data as a rough estimate of the total gross income of the movie.

We then splitted our initial dataset reserving 25% of records for testing and conducted the rest of our analysis exclusively on the training set, as to prevent data leakage.

6.1.1 *Univariate analysis of variables*

The distribution of number of votes, awards, nominations and total gross income show all a pronounced right skew, which is not unexpected. 90% of records had received awards of nominations. All records display normal runtimes for movies (between 90 and 200 minutes), except for 3 documentaries which were shorter than 45 minutes. We were able to detect some outliers also w.r.t. the release year: only two titles were released before 2010 (one in 1969 and one in 1984).

As for the distribution with respect to genres, any given movie can have more than one genre and the most popular genre was *Drama* (48%), followed by *Comedy* (36%), *Action* and *Adventure* (each of these 32%).

We also inspected the distribution of the binned version of rating, finding a considerable unbalance among classes, with *Medium* and *High* being the most frequent ones and the *Low* class representing less than 1% of titles (cf Fig. 25).

6.1.2 *Multivariate analysis of variables*

Figure 26 reports the Spearman correlation coefficient for pairs of numeric features. Besides obviously being correlated with each other, both awards and nominations are positively correlated with both the number of votes and rating. We also observe a moderate to strong positive correlation between the estimated gross income and number of votes. Perhaps surprisingly, total gross income is only weakly correlated with rating.

Positive correlation of the number votes with rating is something we didn't observe in the IMDB dataset, were, on the contrary these two features were weakly negatively correlated. We think this may be due to the fact that movies have on average

1 https://www.boxofficemojo.com/

2 https://imdbapi.dev/

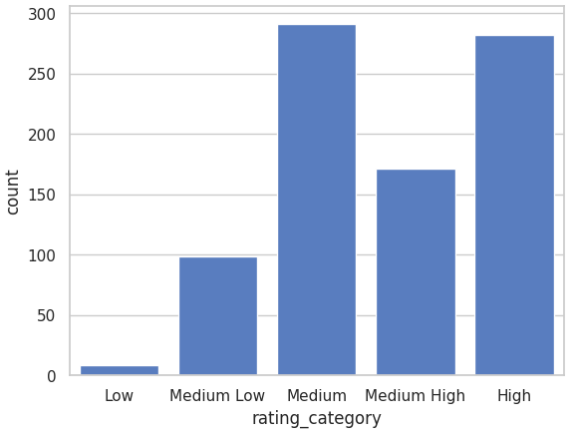


Figure 25: Distribution of rating category in the Time Series dataset.

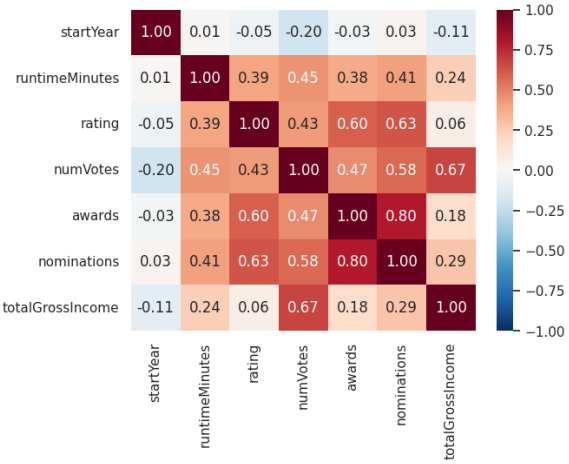


Figure 26: Correlation of numeric features.

lower ratings than TV Episodes, which, in turn, have fewer votes than movies. Since the current dataset is limited to movies, we do not observe this (spurious) relationship anymore.

6.1.3 Visualization

We conclude our understanding visualizing the signals for all time series after amplitude scaling (fig. 27). Even from this simple visualization we are able to draw some interesting insights.

First of all our time series in general show slightly negative trend and some seasonality with a cycle over around 14 timestamps: this observation was confirmed when we extracted Catch22 features from the time series and were able to verify that most of them had a periodicity of 13 or 14. This finding, which we initially took at face value, was somewhat puzzling, because we expected a day of week seasonality, with peaks in the weekends, while it made no sense for peaks

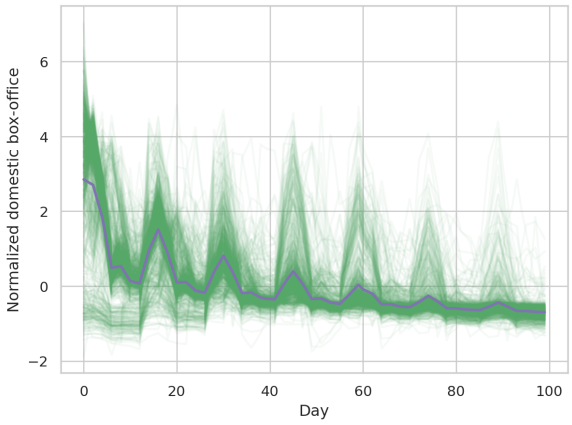


Figure 27: Time series (and mean time series, in purple) after amplitude scaling

to occur every other week. Surely enough, when we finally checked against the original data on Box Office Mojo, we were able to confirm that the observed peaks are, in fact, the effect of day of week seasonality, and that the time series were altered before we received them, either during the extraction or during the preprocessing.

In addition to this discovery, we also were able to observe that, while most movies have profits well over the mean in the first several days after release, some others got a slow start at the box office. We were able to identify 145 movies that performed below average in the first day of box office. While this is a very coarse distinction, which incurs in some false positives, it was later confirmed by clustering. In addition to lower box office profits in the first days after release, these 'late bloomers' show no clear linear trend (in opposition to the negative trend observed for other movies) and tend to peak on the third weekend after release. We also observed that these were not, for the most part (93%), action movies.

6.2 Clustering

In our clustering analysis of time series we adopted two different approaches to the measure of similarity: shape-based similarity and structural similarity.

For the shape-based approach, we first subjected the time series to amplitude scaling and then measured the distances between records employing DTW as to minimize the impact of misaligned time series. Since our time series were very short, we didn't deem it necessary to apply any approximation method to the original signal; we did, however impose constraints (Sakoe-Chiba Band with radius 0.1) on the warping in order to

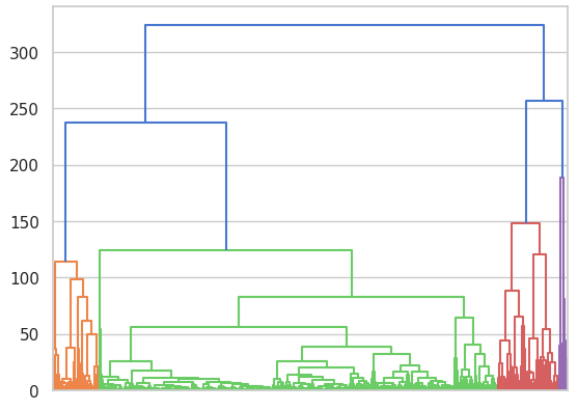


Figure 28: Dendrogram for shape-based hierarhical clustering.

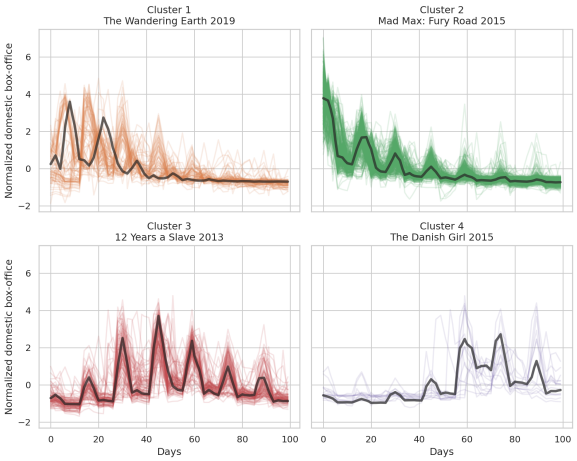


Figure 29: Clusters obtained by hierarchical clustering and respective cluster medoids (in black).

reduce computation time and avoid pathological warpings.

For the global structural approach, we extracted mean, standard deviation and Catch22 features from the raw signals (without amplitude scaling) and then computed Euclidean Distance on this new feature matrix (which was normalized using a Standard Scaler).

6.2.1 *Shape-based clustering*

Since in our understanding phase we were able to recognize two distinct patterns, we first of all run K-Means with k=2 and obtained results very similar to those previously discussed. This clustering obtained a silhouette score of 0.85 and was able to identify 159 'late bloomers'.

We then experimented with different linkage criteria for hierarchical clustering, selecting in the end average linkage, which yielded the best dendrogram (Fig. 28). After selecting an appropriate threshold, we obtained 4 clusters of very different sizes (C1: 75, C2: 658, C3: 101, C4:15). This clustering obtained a silhouette score of 0.77.

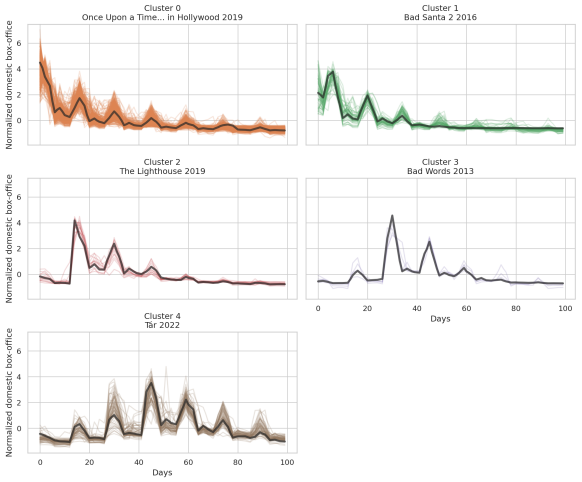


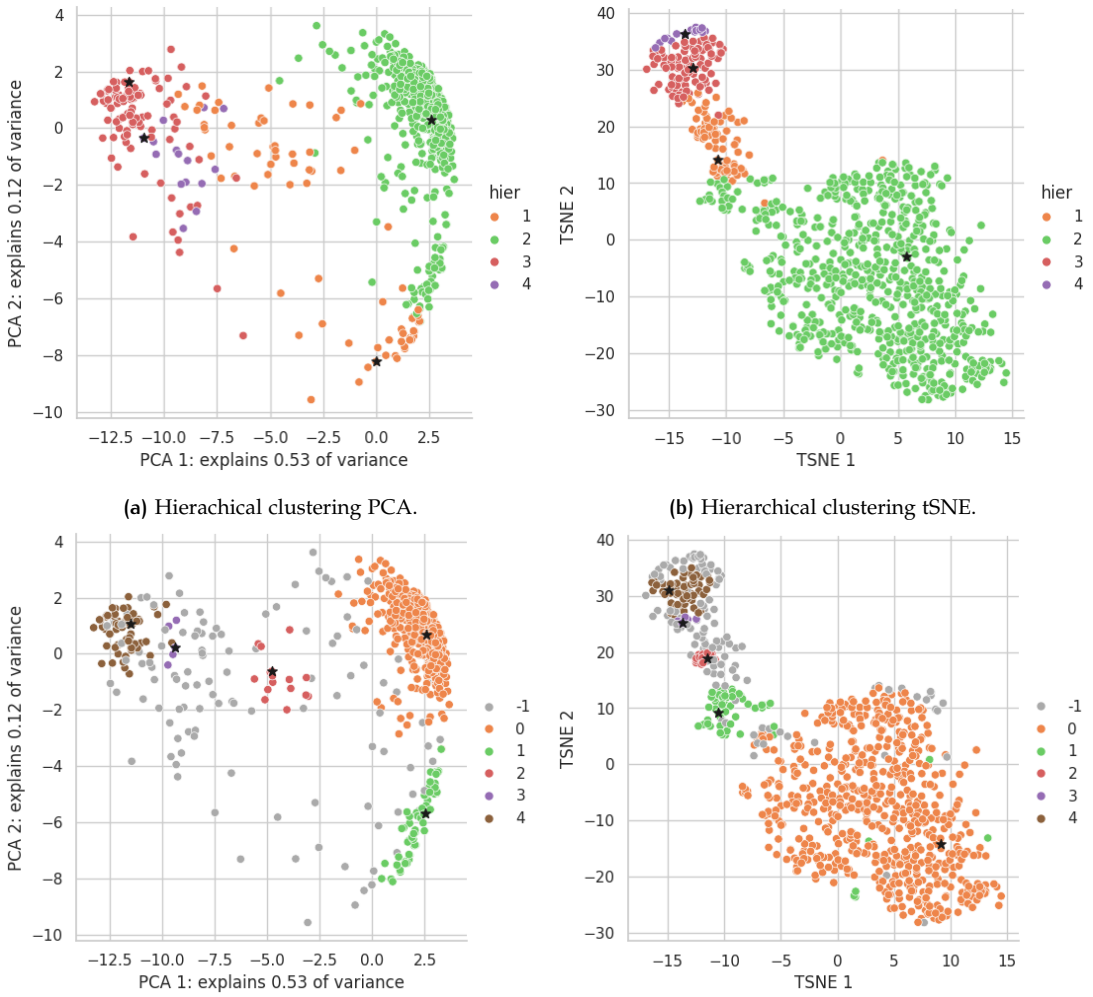
Figure 30: Clusters obtained by density-based clustering and respective cluster medoids (in black).

Signals for clustered time series and cluster medoids are visualized in Fig. 29: it can be easily seen that, while C2 and C3 can be identified respectively with instant hits and 'late bloomers', the identity of movies from C1 and C4 is more confused. Detailed analysis of silhouette scores revealed that many records from C4 and especially C1 have negative silhouette scores, suggesting that they were assigned to the wrong cluster.

We then run Principal Component Analysis on the dataset and visualized clusters on the two first components, which together explain more than 60% of variance (Fig. 31a, cluster medoids are showed in black).

Inspecting the PCA Eigenvectors, we found out that the first component is mainly concerned with timestamps from the first week or so, which explains why records from C2 (and some records from C1) have positive values in this component, while the 'late bloomers' from C3 have very negative values. Also, observing the distribution of records from C1 along the first component, it is clear why more than half of them have a negative silhouette score, since they clearly should belong to C2. In contrast, K-Means, clearly separates the two clusters along the first component.

We also visualized the clusters using tSNE on the DTW distance matrix. We choose this projection for its ability of preserving the local structure of the dataset. While tSNE was able to convey some information on the numeric relative dimensions of different clusters, and was able to visualize clusters nicely, in the end it proved less interesting, because of its inability of revealing the



(a) Hierachical clustering PCA.

(b) Hierarchical clustering tSNE.

(c) DBSCAN clustering PCA.

(d) DBSCAN clustering tSNE.

Figure 31: Clusters and cluster medoids for hierarchical clustering and DBSCAN visualized with two dimensionality reduction techniques.

global structure of the dataset and of conveying the variable density of our clusters.

Inspection of the PCA visualization convinced us that extraction of clusters with a density-based method was viable. Since a series of run with different radii failed to return any interesting clusters, we decided to pursue a different approach: we run OPTICS (with `minPts=4`) to completion on a reduced version of the dataset, using the first 5 components created by PCA (which capture more than 80% of variance). We then visualized the reachability plot and choose an appropriate radius with which to run DBSCAN using the reachability distances computed by OPTICS. DBSCAN detected 116 noise points and returned five clusters of varying sizes (C0: 594, C1:57, C2: 16, C3: 5 and C4: 62, see Fig. 30).

We computed the silhouette score for non noise points on the DTW distance matrix, in order to have results comparable with the other clustering methods. We found that, while globally the silhouette was a little worse than for the previous methods (0.69), essentially no record showed signs of being misclassified.

Inspection of the PCA visualization (Fig. 31c) shows that this method doesn't fall into the pitfalls of the hierarchical clustering, for example finding a good separation between C0 and C1.

However, while C0 and C1 are fully satisfactory, respectively clustering together instant hits and movies with a similar pattern to instant hits, but with a less explosive first weekend, it looks to us that the clustering of records with lower values on the first PCA component is more contentious because of DBSCAN's inability to detect clusters of different densities. Arguably the 'late bloomers' from C3 and C4 should be clustered together (probably with some noise points). C2, despite its size, captured our interest, because it was able to detect a pattern no other clustering

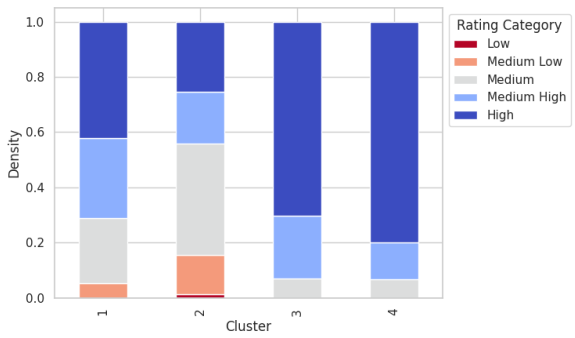


Figure 32: Distribution of binned rating by cluster (hierarchical).

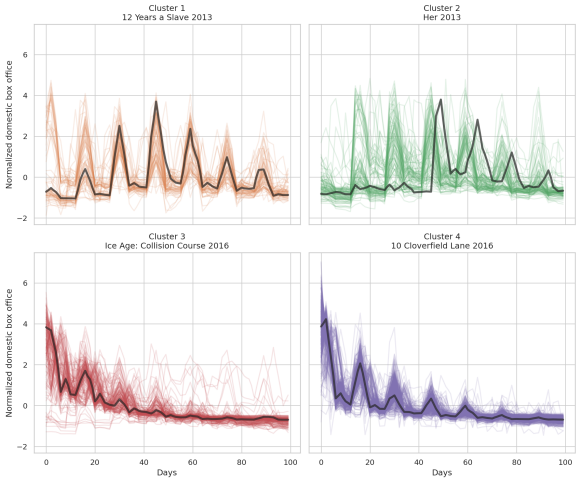


Figure 33: Clusters obtained by hierarchical clustering on structural features and respective cluster medoids (in black).

method was able to capture. It seems to us that movies in this cluster have a slow start at the box office, but, differently from our 'late bloomers', they explode on the first weekend after the release and then display a behavior very similar to instant hits. We hypothesized that this cluster was grouping together movies that were pre-released in a limited fashion.

6.2.2 *Distribution of shape-based clusters w.r.t. exogenous features*

In this analysis we concentrated our attention on instant hits from C3 and 'late bloomers' from C2 returned by the hierarchical clustering. We noticed that 'late bloomers' have on average a lower average box office (median in the 100k) than instant hits (median in the millions). This is a valuable finding, since we applied amplitude scaling on the time series before analysis.

In addition to that, we found that 'late bloomers' are treated more favorably in terms of rating, as can be seen from and receive on average more awards and nominations. Action movies are severely underrepresented among 'late bloomers' (only 5% vs 38% of records from C2 and 32% globally), while dramatic movies are overrepresented, since, despite making up about half of the dataset, they make up more than 85% of C3.

6.2.3 *Global structural features clustering*

We ran hierarchical clustering on time series structural features experimenting with different linkage criteria and selecting the one that gave use the best dendrogram (Ward). We obtained four clusters and a mean silhouette of 0.21 (it should be noted that we do not invite comparison with the silhouette obtained by shape-based methods, since this clustering was performed on an entirely different distance matrix).

From Fig. 33 we can see that also this clustering method was able to detect the patterns observed previously, albeit in a less marked way. C1 and C2 (which are later merged in a single cluster) are mainly made up of 'late bloomers', while C3 and C4 group together instant hits (C3 and C4 are sisters).

Analysis of clusters w.r.t. exogenous features showed very similar patters to those already observed in terms of rating, binned rating, awards, nominations, distribution of action and dramatic movies.

6.3 Motifs, discords and shapelets

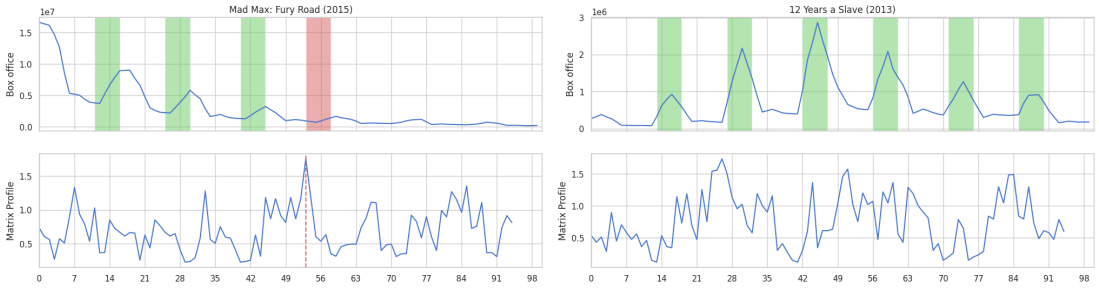
In this section we performed several analysis related to motifs and discords in our time series leveraging the Matrix Profile.

6.3.1 *Motifs and conserved patterns*

Our first analysis was extracting and visualizing motifs on the cluster medoids found in 6.2. Knowing that our time series display day of week seasonality (and therefore they have a period of around 14 timestamps), we inspected results for different window sizes, before settling on 6.

Fig 34 shows in green the motifs found on two cluster medoids that are good representative, respectively of instant hits and 'late bloomers'.

Additionally, we looked for a conserved pattern between these two medoids. Interestingly, the best matching pattern visualized in 35 is also a motif in *Mad Max: Fury Road* (see again fig 34a). This may suggest that the seasonal peaks in movies from the two clusters are not so different after all and



(a) Centroid of C2. **(b)** Centroid of C3.

Figure 34: Motifs (and discord) discovered on cluster medoids.

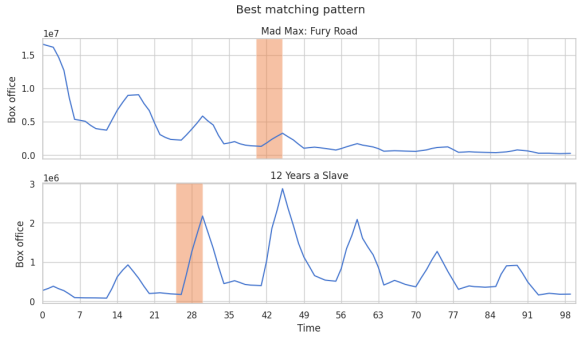


Figure 35: Best matching pattern between cluster medoids.

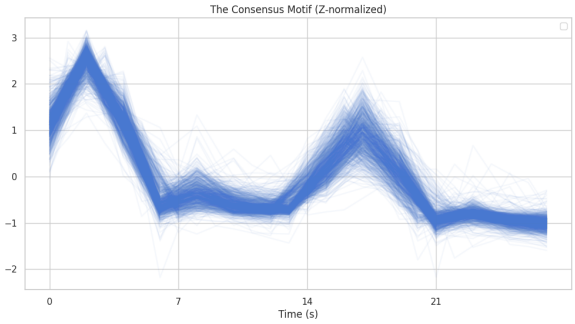


Figure 36: Best match for the consensus motif on all time series.

probably reflect a general pattern of movie-goers behavior.

6.3.2 *Discords*

In order to detect discords, we searched for points on the matrix profile that were deemed anomalous by a one-sided Grubbs test (one such subsequence is reported in red in fig. 34a). With our more stringent definition of discord, we were able to detect 89 records in our dataset that contained anomalous patterns.

Interestingly, the two records that we identified as outliers w.r.t. year of release, were indeed in this group. When we visualized them, we noticed that they bear scarce resemblance to the time series analyzed so far. This may warn us that our analysis isn't really generalizable to longer box office trends out of the specific historical period represented in our dataset, as it is predicated on societal trends that may have less staying power than we may think.

6.3.3 *Consensus Motif*

We also leveraged the matrix profile in order to find a consensus motif, i.e. the subsequence from the entire dataset that induces the smallest radius. We ran the *Ostinato* algorithm on our data with a window size of 28 timestamps (i.e. a two weeks period) which we judged to be good size for finding a discriminative subsequence.

From each time series we extracted the best match for the consensus motif (see fig. 36). The visualization of the matching subsequences showed a pattern that is well known to us and this point: two major peaks in the weekend (the second smaller than the first) and smaller peaks in the middle of the week after each major peak. For most time series we observed only small deviations from this motif: for example a subtle anticipation of the second major peak or a relatively higher midweek peak.

We decided to perform clustering on this novel representation of our time series. Hierarchical clustering failed to yield any results comparable with those obtained by shape based or feature based clustering.

Also running OPTICS and inspecting the reachability plot didn't allow us to discover any clusters. However, running DBSCAN on the reachability distances computed by OPTICS with an appropriate value for ϵ (chosen in correspondence of an elbow in the reachability plot) returned one single cluster and 82 noise points. Inspecting the distribution of noise points w.r.t. known features

revealed that movies that we had deemed anomalous either by release year or by runtime during the understanding phase were classified as noise by the density based clustering.

While some series conserve the consensus motif with less fidelity (or arguably they do not conserve it at all), it wasn't possible to detect an interesting pattern in the variation of the motif. This probably makes sense, considering the nature of our data: while the shape of the entire time series has enabled us to find groups of movies with distinct patterns (especially with respect to the position of the highest peak and the general trend of the time series), this conserved motif is probably registering a common pattern in the behavior of movie goers, which is largely independent of the characteristics of the specific movie.

6.3.4 Shapelet discovery

In this section we leveraged the matrix profile in order to extract shapelets that would be of use in predicting if a movie is an action movie or not.

To do this, we split our training data in action and non action movies and, after amplitude scaling, concatenated all the individual samples of each class in one long time series. In order to avoid finding spurious patterns across time series, we established a clear boundary, appending `Nan` at the end of each time series.

The basic idea is that the matrix profile $P_{Action,Action}$ computed on the time series T_{Action} allows us to identify conserved subsequences within T_{Action}. A matrix profile $P_{Action,NoAction}$, computed across two different time series T_{Action} and $T_{NonAction}$ compares all the subsequences of T_{Action} with all the subsequences in $T_{NoAction}$ in order to determine if there are any subsequences in T_{Action} that can also be found in $T_{NoAction}$. If a discriminative pattern (shapelet) is present in the *Action* class but not in the *NoAction* class, then we would expect to see significant differences in heights among the matrix profiles $P_{Action,Action}$ and $P_{Action,NonAction}$. So, if we compute $P_{diff} = P_{Action,NoAction} - P_{Action,Action}$ we expect peak values to indicate good shapelets candidates, because they suggest patterns that are well conserved in their own class but very different from their closest match in the other class.

After computing the matrix profiles with a window of 28 timestamps (so on a period of two weeks), for each class we extracted the top 10 candidate shapelets, which we then used in the classification task as detailed in 6.4.4.

6.4 Classification and regression

For this section, we defined two classification tasks and one regression task to be approached with different representations of the time series.

The first is a binary classification task that aims to detect action movies in our dataset; for the evaluation of this task we targeted the F1 score for the positive class, as we were interested in achieving a balance between precision and recall (see table 8 for results in validation).

The second classification task we undertook is a multi-class problem consisting in predicting the binned rating of a movie; since, as we already noticed in 6.1, rating classes are imbalanced, we evaluated model performance on the basis of the macro averaged F1 score, as to guarantee acceptable performances on all classes (see table 9).

In addition to these tasks, we also experimented with some models on the regression task of predicting the mean rating of a movie (see table 10).

In order to establish that our classifiers performance was better than random, we compared results with those obtained by a dummy classifier with stratified strategy; similarly, for regression, we checked that the MSE of the models didn't exceed the variance of the target feature (0.82). All models employed performed much better than their respective baselines.

6.4.1 Distance-based approaches

We employed KNN classifiers (and regressor) on the signal of our time series. Since no approximation was needed, the only preprocessing step performed on the raw signals was amplitude scaling.

For each task, we conducted grid search in order to find the optimal configuration of hyperparameters: we tested different values of k between 1 and 60 and whether to weight exemplar by distance. But the hyperparameter we were most interested in was the distance function: we considered both Euclidean Distance and two versions of constrained DTW (Sakoe-Chiba Band of radius 0.05 and 0.1). We evaluated the performance of each configuration of hyperparameters performing a 5-fold stratified cross validation on the training set.

In our pursuit for the best parameters, we considered area under the PR curve for the binary classification task, macro averaged F1 for the multi-class problem and the coefficient of determination for regression. Overall our models demonstrated great sensibility to the number of neighbors, either plateauing or decreasing perfor-

Model	precision	recall	f1	PR auc
baseline	.30 ± .06	.30 ± .05	.30 ± .05	.32 ± .02
KNN	.56 ± .02	.52 ± .04	.54 ± .02	.58 ± .03
C22 RF	.65 ± .05	.42 ± .05	.51 ± .05	.63 ± .05
TSF	.63 ± .05	.41 ± .03	.50 ± .03	.62 ± .05
drCIF	.66 ± .04	.41 ± .08	.50 ± .06	.58 ± .05
RST	.62 ± .03	.43 ± .05	.51 ± .4	.59 ± .10
Shapelets MP df	.45 ± .04	.46 ± .08	.45 ± .06	.40 ± .04
Shapelets MP rf	.60 ± .09	.34 ± .07	.43 ± .05	.55 ± .05
Proximity Forest	.68	.40	.50	.60

Table 8: Performance of tested models for the action classification task in validation (mean and standard deviation).

Model	accuracy	precision (macro)	recall (macro)	f1 (macro)
baseline	.30 ± .02	.21 ± .01	.21 ± .01	.21 ± .01
KNN	.43 ± .03	.33 ± .06	.28 ± .02	.27 ± .02
C22 RF	.45 ± .04	.32 ± .05	.30 ± .03	.28 ± .03
TSF	.43 ± .04	.31 ± .04	.29 ± .03	.29 ± .03
drCIF	.46 ± .05	.35 ± .04	.32 ± .03	.31 ± .03
Proximity Forest	.42	.29	.27	.26

Table 9: Performance of tested models for the rating classification task in validation (macro averages).

Model	mean squared error	R2
KNN	.69 ± .11	.14 ± .07
C22 RF	.58 ± .08	.27 ± 0.9
TSF	.62 ± .01	.22 ± .08
drCIF	.60 ± .06	.23 ± .13

Table 10: Performance of tested model for the regression task.

mance between 10 and 20 neighbors and generally performed better when exemplars were weighted by distance.

Impact of the presence of warping and warping constraints was generally negligible, with euclidean distance performing better on the binary classification task. Even when warping impacted positively the model performance (for the multiclass problem), the improvement was negligible when considering the score variance across folds. This is probably evidence that our time series are generally well aligned, which is both evident by visualizations of the signal and consistent with our domain knowledge: as a matter of fact, considering that these time series show day-of-week seasonality and that movies are most often released on Friday, we would expect major peaks and valleys to be reasonably in sync.

When evaluating our models on 5-fold stratified cross validation, we found that all of them were able to clear the respective baselines without problems. In particular, even without tuning the decision threshold our classifier obtained precision, recall and f1 all over 0.5 on the binary classification task.

The regressor demonstrated no sensibility at all to distance or weights parameters, so we picked the simplest configuration possible.

We also experimented with a second distance based approach: Proximity Forest is a distance based classifier that creates an ensemble of decision trees where splits are based on the similarity of the candidate time series with two exemplars measured using various distance measures. Since this model took much longer for fitting than the previous ones, we didn't tune parameters and performed holdout validation instead of 5-fold cross validation, reserving 25% of the training test for validation.

For both classification tasks, performance was comparable with that of KNN.

6.4.2 Global structural features approaches

Our second approach was to train a classifier on global structural features extracted from the original signals. We used the same set of features we already performed clustering on (see 6.2.3) and used Random Forest Classifier/Regressor as an estimator, a model we chose for its ability to provide somewhat interpretable results (by inspec-

tion of feature importance) while being less prone to overfitting than Decision Trees.

When tuning hyperparameters, we let individual trees fully develop and manipulated only the number of estimators and the maximum number of features used by each tree: we experimented with 10, 50, 100 and 200 estimators and with the following values for the maximum number of features: `log2`, `sqrt`, all features. Grid search was performed with the same modalities and target scores detailed in 6.4.1.

While 10 estimators proved too few, overall the models didn't show great sensitivity to hyperparameters: standard deviation across folds was generally greater than differences between parameters configurations. For both multi-class classification and regression, 50 estimators and all features were found to be the best configuration of parameters, while for binary classification we used 200 estimators and `max_features=log2`.

On cross-validation these models outperformed KNN on both classification tasks, albeit non decisively, but not on the regression task. While F1 score on the binary classification task was lower than that of KNN, area under the PR curve was greater, indicating that KNN could be out-performed with decision threshold tuning.

6.4.3 *Ensemble of interval-based approaches*

These approaches operate on representations of the original time series that are created by extracting summary statistics from selected subsequences. We experimented with Time Series Forest, a very simple model that summarizes intervals only by mean, standard deviation and slope, and with Diverse Representation CIF (DrCIF), which represents an extension of Canonical Interval Forest and is generally considered the state of the art for this family of approaches.

For both TSF and drCIF, we experimented with different combinations of minimum and maximum interval length. For the minimum interval length we capitalized on insights gained from motif discovery, so we tested with lengths of 6, 7, 8, 14, 28 and 35 timestamps. Generally speaking, a smaller interval length (7) was preferred for all tasks. As for the maximum interval length, we allowed for either half the size of the time series, or the whole time-series. We noticed that the influence of this second parameter became non-existent the more estimators we considered: for TSF we experimented with 5, 10, 50, 100 and 200 estimators, while for drCIF, which is much more computationally expensive, we limited the search to 5 and 10 estimators. While the impact

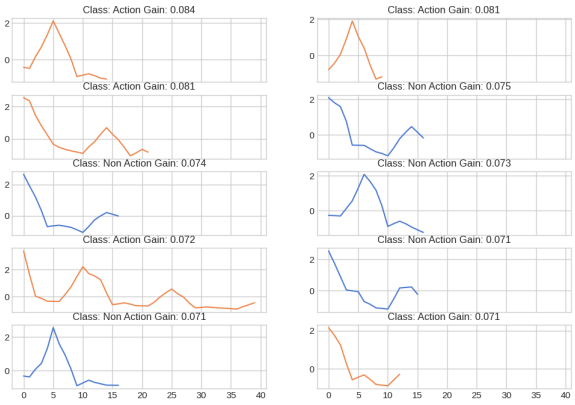


Figure 37: Shapelets extracted by Random Shapelet Transformer (shapelets for the positive class in orange).

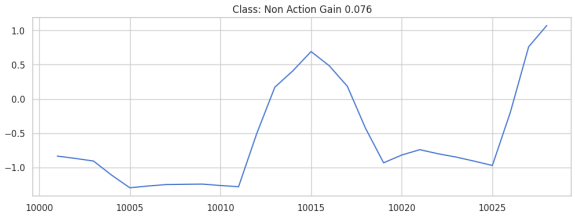


Figure 38: Shapelet extracted from Matrix Profile.

of number of estimators is not clear on the binary classification task, we noticed improvements on both the multi-class problem and the regression when using more estimators.

On 5-fold cross validation, TSF and drCIF demonstrated performances comparable to those of the previous approach. The biggest gap is on the regression task, but considering the variance across scores from different folds, this difference is probably not significant. It is possible that drCIF performance, which is practically indistinguishable from that of TSF, was hindered by our choice to limit the number of estimators considered.

6.4.4 *Shapelets*

At last, we approached the binary classification task with two different approaches to shapelet-based classification.

In the first case we used a random shapelet transformer in order to extract the 10 most discriminative shapelets from 1000 random candidate subsequences, and trained a random forest on the representation returned from the transformer. For the transformer, we experimented with different maximum shapelet lengths, that were informed by previous analysis on motifs and our experience with interval based classifiers. We run a grid search on maximum length 14, 28,

50 and the degenerate case of accepting the entire time series as a subsequence. We found that a maximum length of 50 returned the best results in terms of average precision. In cross validation this classifier was the second best after the classifier based on Catch22 features, with an average precision of 0.62.

We also analyzed the extracted shapelets, that are visualized at fig. 37. We can notice that most of them have a length around 14 timestamps and as such they are able to capture patterns occurring in the course of a week. On the other hand it can be somewhat difficult to interpret the subtle distinctions that make these shapeletes particularly discriminative for the current task. For example: three of the extracted shapelets show a pretty similar pattern, with a major weekend peak around the sixth timestamp and a minor peak some time later, certainly to be attributed to people that go to the movie theater on Tuesday in order to beat the crowd and get discounted tickets. However, one of these is associated with the positive class, while the other two with the negative class. While the general context for these patterns is clear, the subtle differences that make them contribute to a correct classification escape us.

Our second approach was to leverage the 20 shapelets extracted from the matrix profile as detailed in 6.3.4.

We initially attempted to find the top shapelets from our initial set using recursive feature elimination to determine the ranking of all features and then testing the same Decision Tree classifier on the top n features in order to determine the optimal selection. However, since the performance with this approach was not very impressive (average precision of 0.4), we decided to retain all candidate shapelets and adopt a Random Forest Classifier. Results obtained with Random Forest were much better (although, not at the levels reached by RST), reaching an average precision of 0.55.

After training the model on the entire test set, we inspected the shapelets with higher importance. The four top shapelets were all extracted from the negative class: this made sense to us, since, as we noticed during clustering, 'late bloomers', that have a very distinctive shape when compared with the rest of the dataset, tend not to be action movies.

Fig 38 shows one of the top shapelets: we can notice two major weekend peaks with a positive trend; this is indeed a very distinctive pattern for 'late bloomers'that doesn't occur in instant hits,

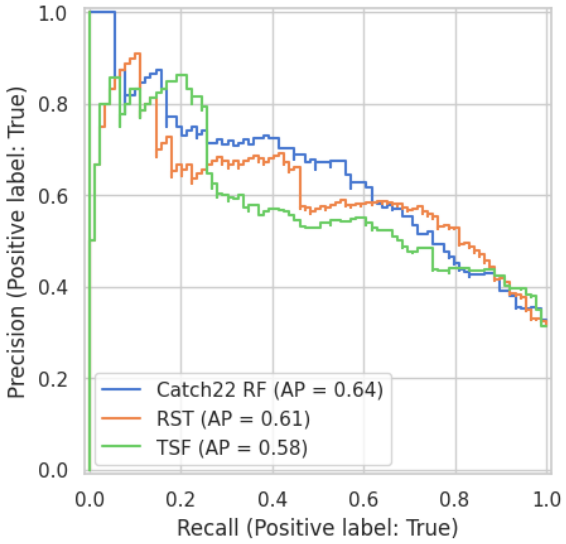


Figure 39: Precision-Recall curves for the action classification task on test set.

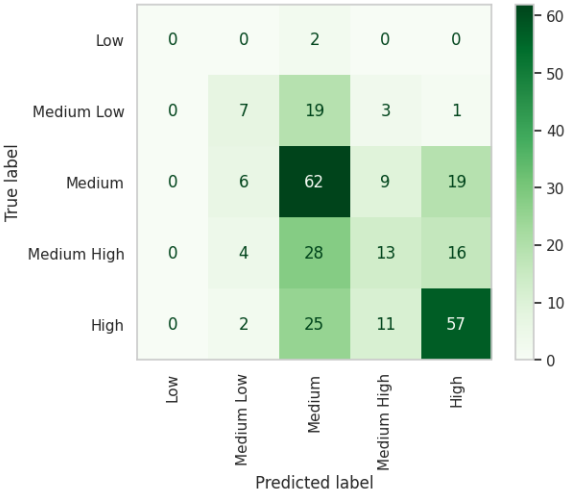


Figure 40: Confusion matrix for C22 classifier on the test set.

that, on the contrary have a negative trend with top profits in the first few days after release.

6.4.5 *Assessment*

For our assessment, we evaluated the performance of the Catch22 RF and of TSF on the test set. For the binary classification task, shapelet based models were also included.

For the binary classification task, we tuned the decision threshold of each model with 5-fold cross validation on the training set before evaluating it on the test set.

Catch22 RF and RSF performed very similarly on the test set, obtaining the same F1 score for the positive class (0.61) albeit with a different trade

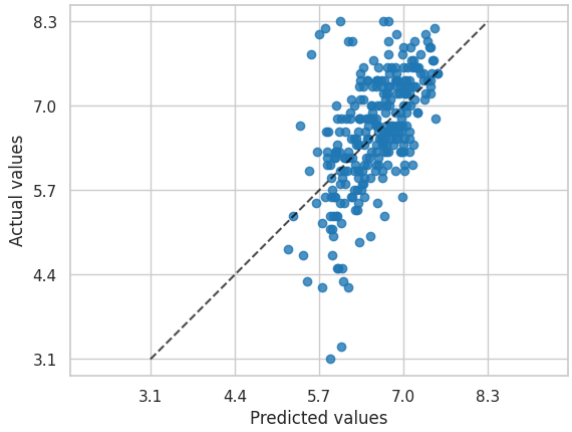


Figure 41: Relationship between real ratings and rating predicted by the C22 regressor during testing.

off between precision and recall (RSF had higher precision). From the PR curves in fig 39 we can observe that, while all considered classifiers have a similar performance, Catch22 RF demonstrated better ranking capabilities, both in terms of average precision and of stability of the PR curve.

We also tested the model that is using manually extracted shapelets, not because it showed impressive performance in validation (it did not), but because we wanted to confirm our hypothesis that this model overfitted the validation set, since we performed shapelet extraction on the entire training set. Surely enough, average precision showed degradation on the test set (decreasing to 0.52), and in particular produced more top ranked false positives than during cross validation.

For the multi-class problem, the best performance came from Catch22 RF, which obtained a macro F1 of 0.34 (0.7 higher than the contender), and showed consistently better performance than TSF on all classes. Inspecting the confusion matrix in fig. 40, we can see that our model performs the best on classes with the highest support (High and Medium), while both Medium High and Medium Low are frequently misclassified with as Medium. Low has 0 recall.

Catch22 RF was also the best regressor: with a mean squared error of 0.55 (much lower than the target variable variance) and a coefficient of determination of 0.34, if clearly outperformed TSF (R2 0.29). From fig. 41 we can see that, although predicted values are squeezed towards the mean, still they display a positive linear correlation with actual values (Pearson R 0.58, p value < 0.01).